Retrospective: what's the right computing model?

Hung-Wei Tseng

Change in the System

Samsung Smart SSD

- **New Smart SSD APIs** to run database operations inside the SSD
- Open(), Get(), Close(), SendToHost()

SQL Server

- Modified the query processor to hard-code selected Smart SSD plans
- Implement Smart SSD executors

PRODUCT BRIEF

AMD XILINX

SmartSSD® COMPUTATIONAL STORAGE DRIVE

OVERVIEW

The Samsung SmartSSD computational storage drive (CSD) is the industry's first adaptable computational storage platform. It empowers a new breed of software developers to easily build innovative hardware-accelerated solutions in familiar high-level languages. The SmartSSD dramatically accelerates data intensive applications by 10X or more by pushing compute to where the data lives.

At the heart of the SmartSSD is the Xilinx Adaptive Platform, a powerful toolkit which enables customers to create customized, scalable applications to solve a broad range of data center problems.

CHALLENGE

Turning Big Data Into Fast Data

The global datasphere will grow from 45 zettabytes in 2019 to 175 by 2025, and nearly 30% of the world's data will need real- time processing. This data explosion provides tremendous opportunity for business insights, but the sheer volume

TARGET APPLICATIONS

- AI/ML Inference
- Big Data Analytics

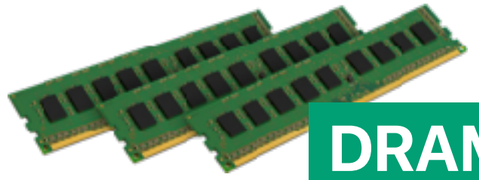
Varifocal Storage: A Holistic System Architecture

Retrieve Raw Data

Adjust Data Resolutions

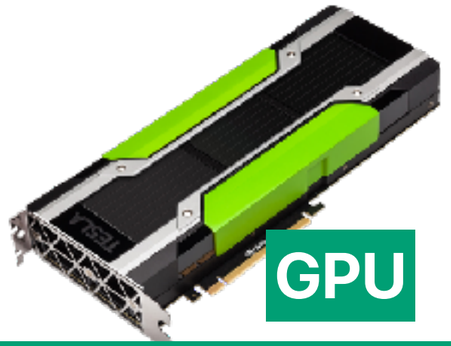
Varifocal Storage (VS)

System Interconnect (e.g., PCIe)

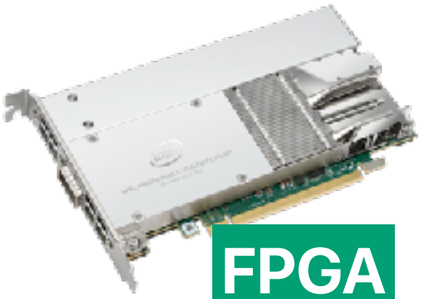


DRAM

CPU



GPU



FPGA

Compute

TPU

Quality Control

(e.g., PCIe)

The "Winning Formula" of In-Storage Processing

Conventional Model

ISP Model

$$\frac{DataVolume_{raw}}{Bandwidth_{device}} + \frac{DataVolume_{raw} \times ComputationIntensity_{NDP}}{ComputationThroughput_{device}} + \frac{DataVolume_{afterNDP}}{Bandwidth_{device2host}} + \frac{DataVolume_{afterNDP} \times ComputationIntensity_{afterNDP}}{ComputationThroughput}$$

$$<= \frac{DataVolume_{raw}}{Bandwidth_{device2host}} + \frac{DataVolume_{raw} \times ComputationIntensity}{ComputationThroughput}$$

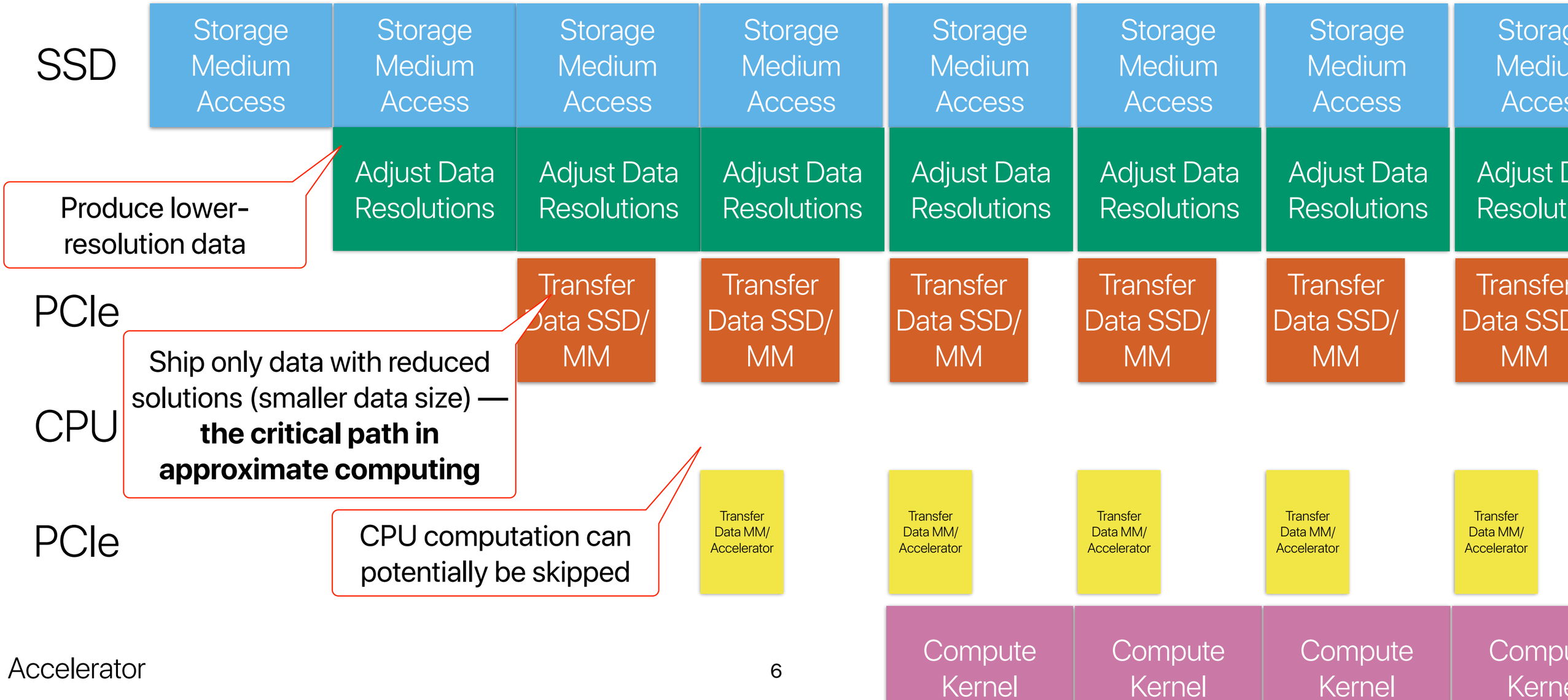
The ISP processor must have efficient internal access to the device

The ISP computation cannot take too long

The ISP should reduce data volume

The ISP should offload the processor

Data processing with ISP



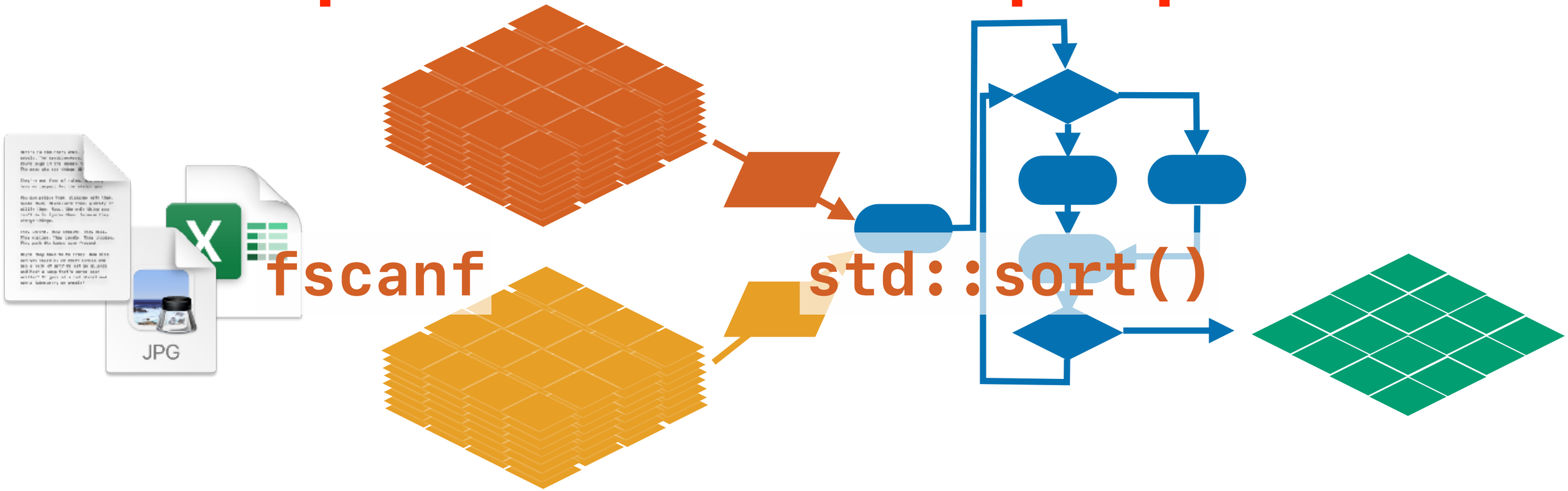
Case: where do you expect the program to spend the most time?

3:00

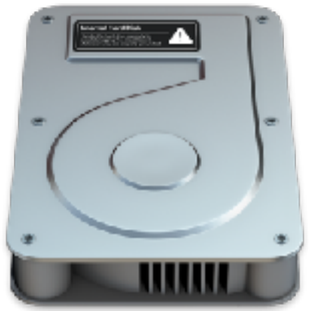
```cpp
#include <stdio.h>
#include <stdlib.h>
#include <algorithm>
#include <ctime>
#include <iostream>
#include <climits>
#include <sys/time.h>

int main(int argc, char **argv)
{
    // input data
    unsigned array_size = 100000000;
    FILE *fp;
    int *data, *sorted;
    int option = atoi(argv[1]);
    double total_time;
    struct timeval time_start, time_end;
    if(argc > 2)
        array_size = atoi(argv[2]);
    long long sum = 0;
    fp = fopen(argv[1],"r");
    data = (int *)malloc(sizeof(int)*array_size);
    sorted = (int *)malloc(sizeof(int)*array_size);

    gettimeofday(&time_start, NULL);
    for(int i=0;i<array_size;i++)
    {
        fscanf(fp, "%d", &data[i]);
    }
    gettimeofday(&time_end, NULL);
    total_time = ((time_end.tv_sec * 1000000 +
time_end.tv_usec) - (time_start.tv_sec * 1000000 +
time_start.tv_usec));
    fprintf(stderr,"Scan Latency: %.0lf us, Goodput
(Integers Per Second): %lf\n",total_time,
array_size*1000000/total_time);

    gettimeofday(&time_start, NULL);
    std::sort(data, data + array_size);
    gettimeofday(&time_end, NULL);
    total_time = ((time_end.tv_sec * 1000000 +
time_end.tv_usec) - (time_start.tv_sec * 1000000 +
time_start.tv_usec));
    fprintf(stderr,"Sort Latency: %.0lf us, Goodput
(Integers Per Second): %lf\n",total_time,
array_size*1000000/total_time);
    fprintf(stderr, "Samping points: data[0: %d, mid: %d,
end: %d]\n", data[0], data[array_size>1],
data[array_size-1]);
    fclose(fp);
    return 0;
}
```

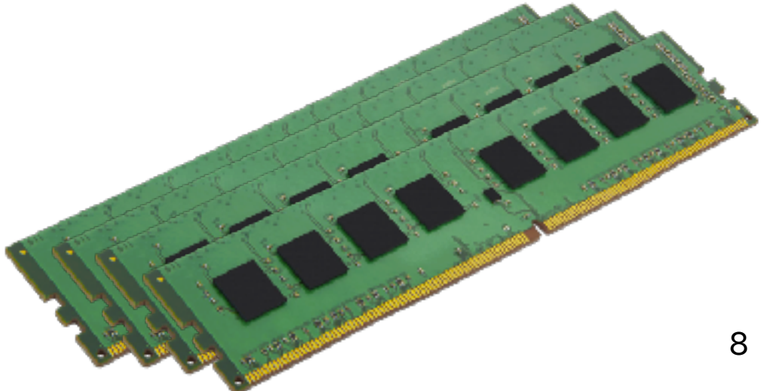
https://github.com/hungweitseng/EE277/blob/main/demo/sort/arraySort.cpp

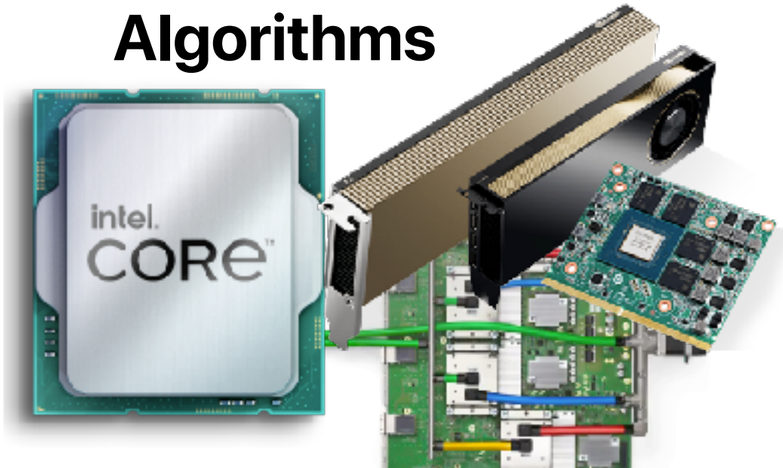
Recap: From the software perspective



Source “data" **“Data” structures** **Algorithms** **Result**







There are lots of data formats

ASCII

“12345678”

⇩

0x3231 0x3433 0x3635 0x3837 0x000a

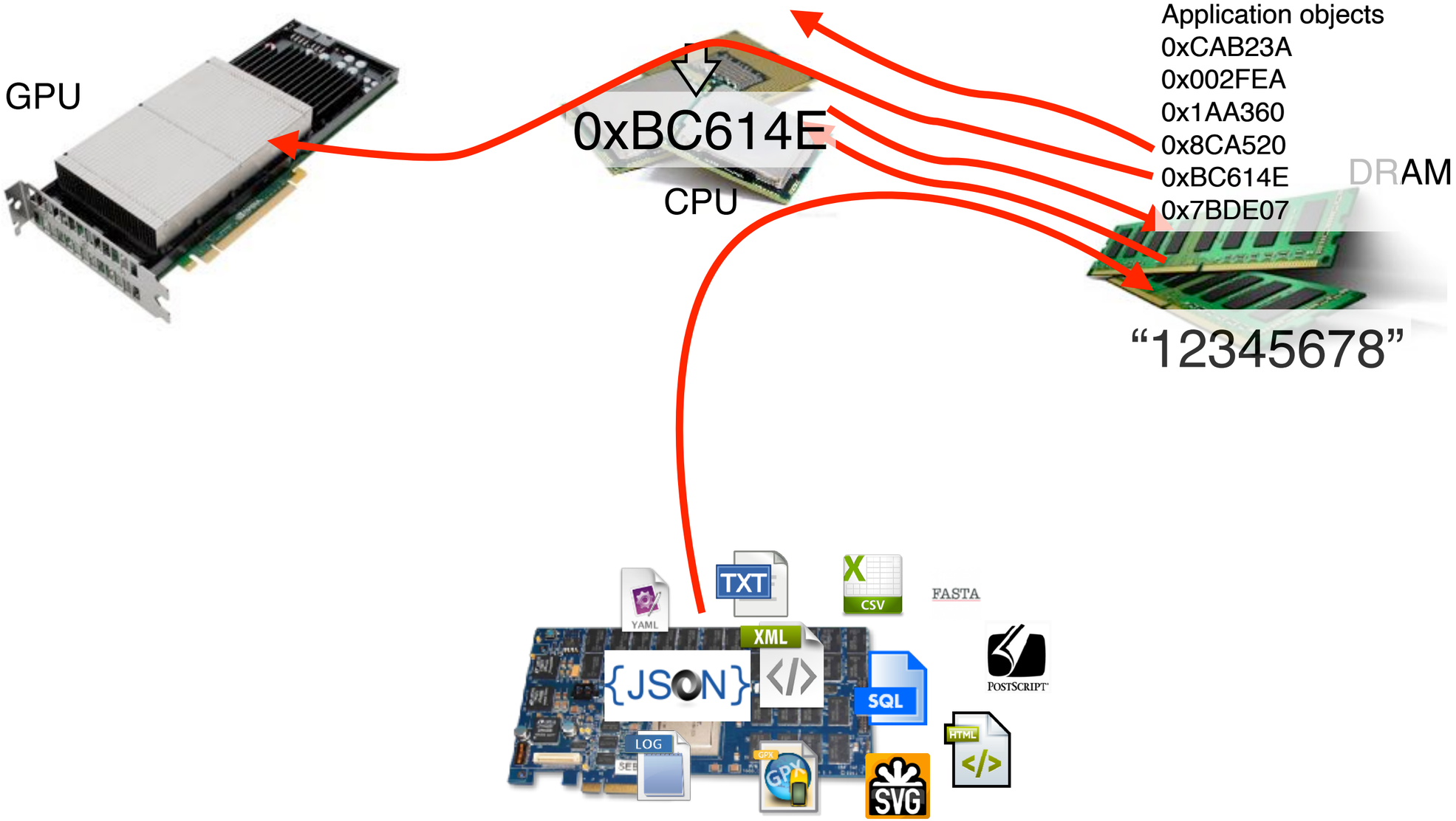
Binary

12345678

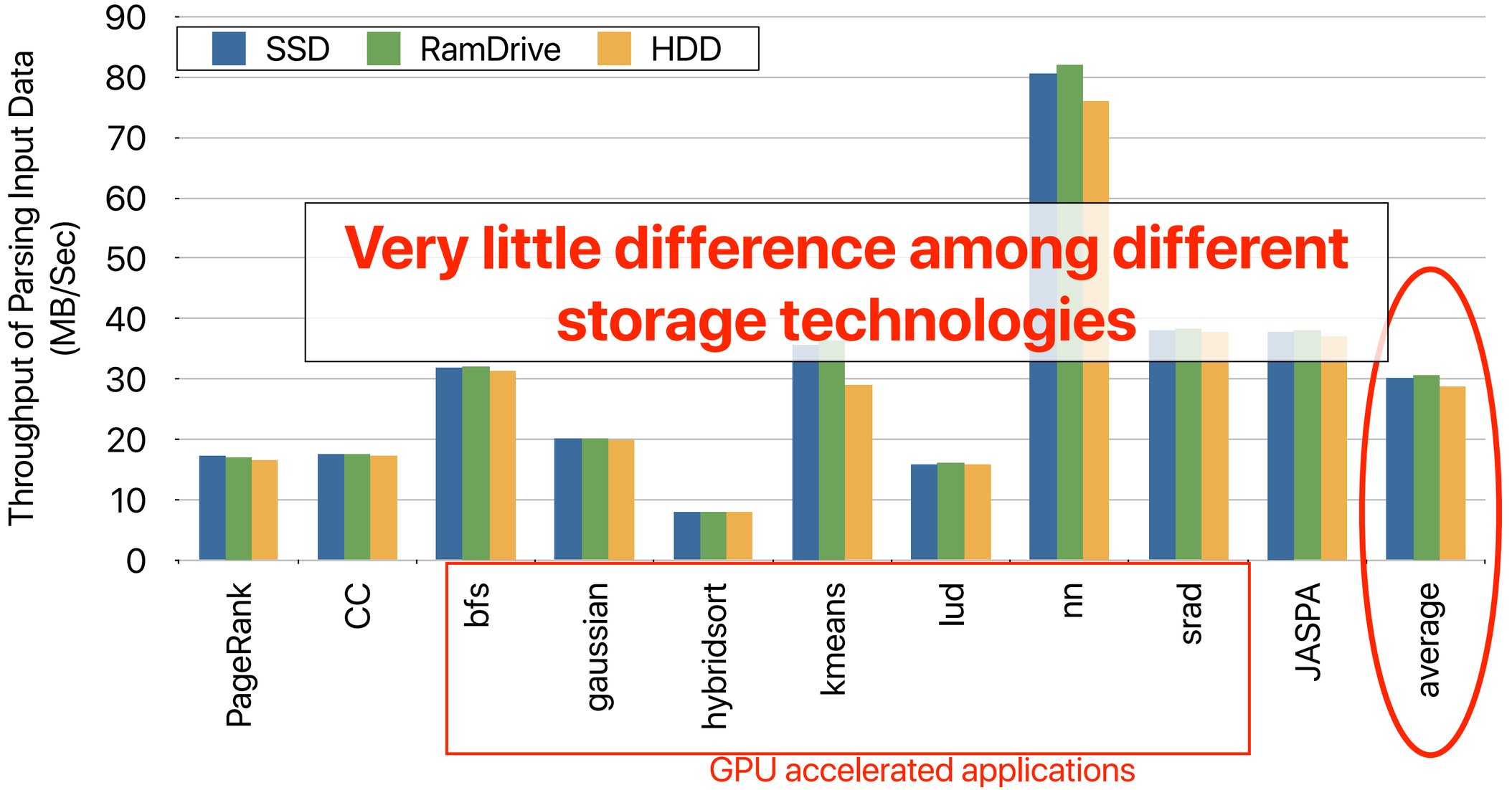
⇩

0xBC614E

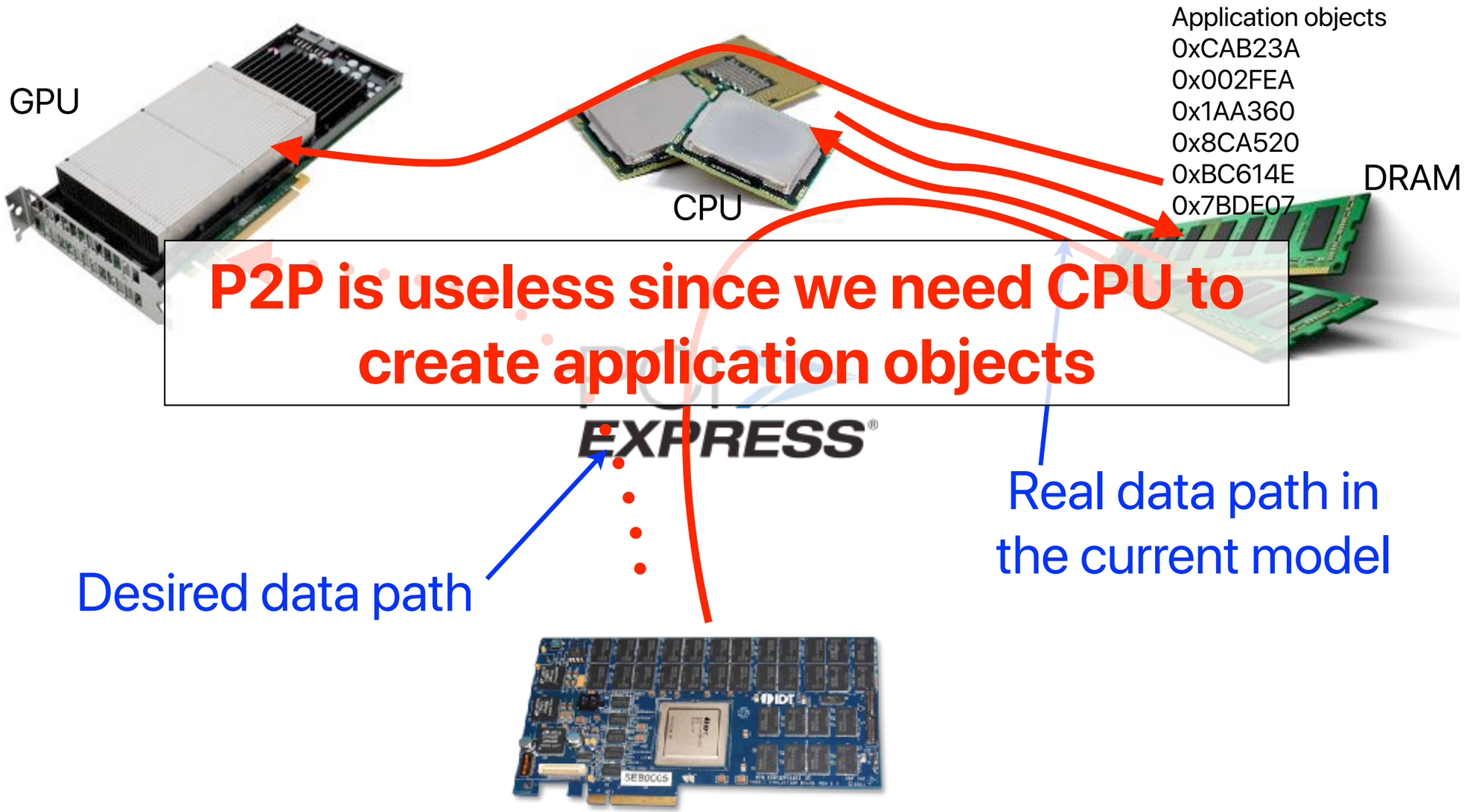
Processing “ASCII” data



High-speed storage is useless



P2P is also useless

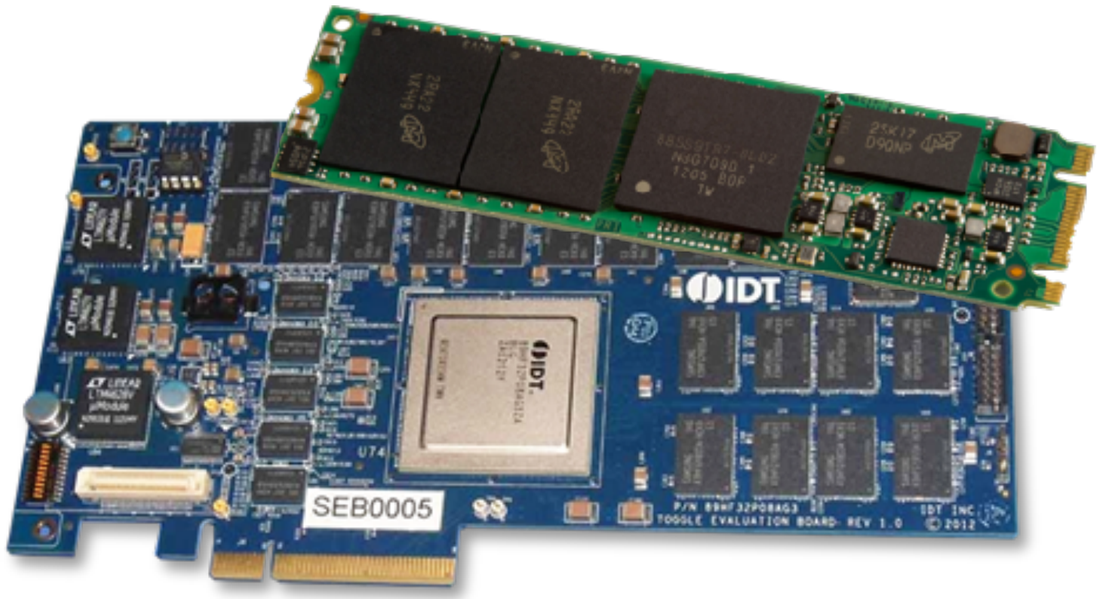


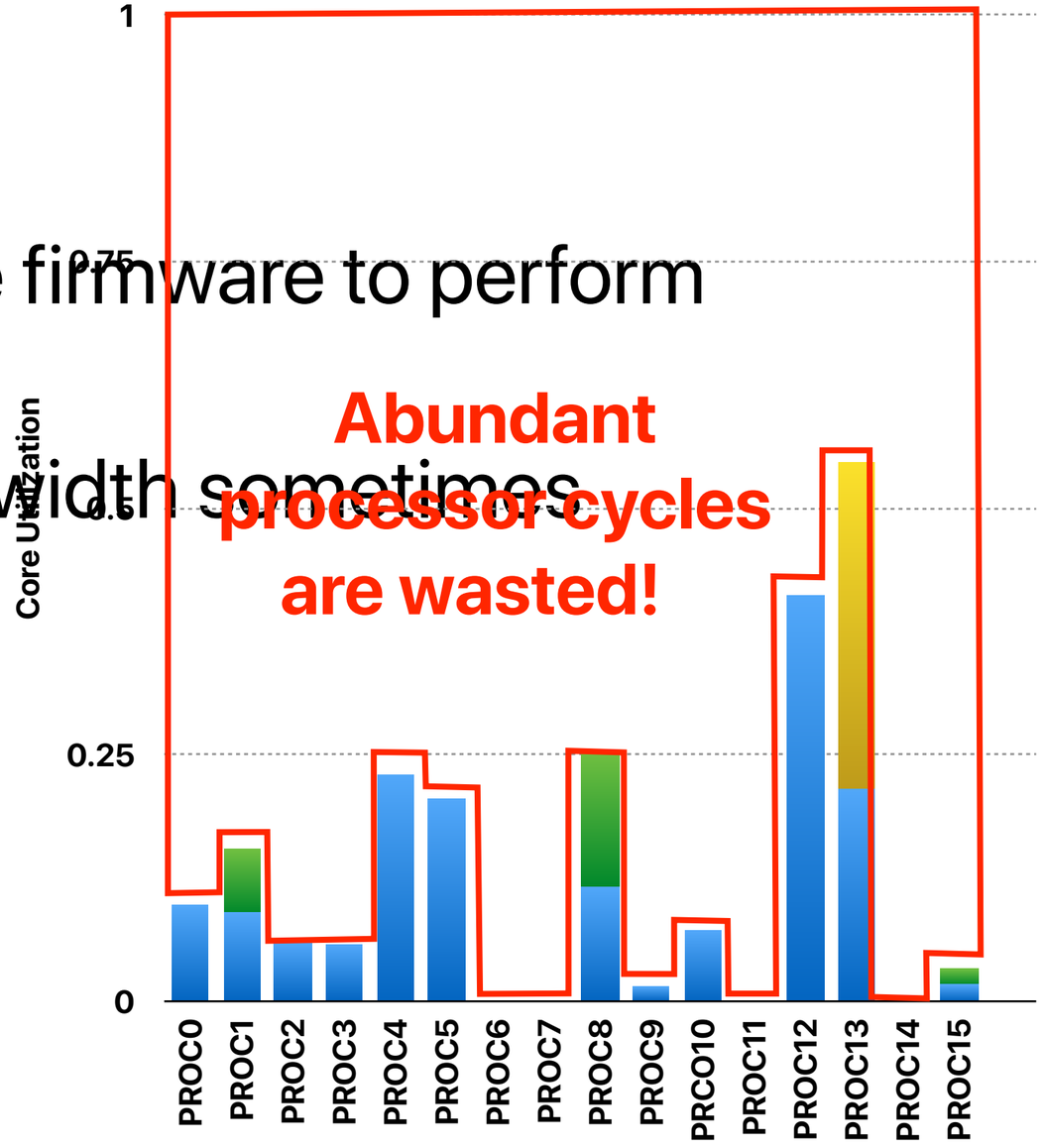
What have you learned?

- Parsing the input takes as long as the computation (or longer if your sort is optimized)
- The good-put of parsing is way lower than the I/O bandwidth
- If you don’t store data in the most appropriate format, it does not matter what kind of storage device you use

Are we using them “efficiently”?

- Firmware does not rely on OS
- Utilization is not optimized as well
- We can “tweak” and “extend” the firmware to perform computation!
- Applications cannot use full bandwidth sometimes

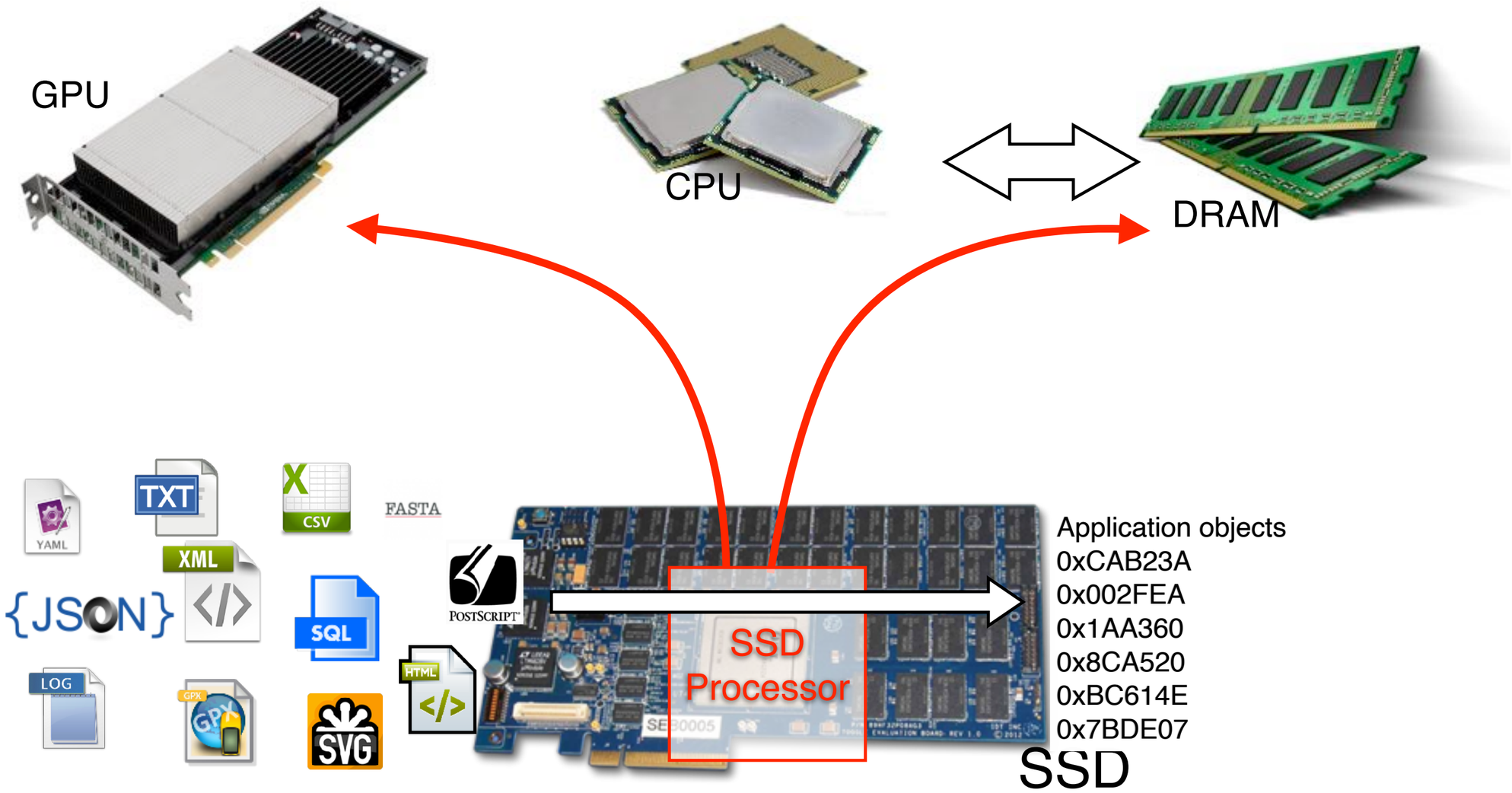




Example — Morpheus✳

✳ Hung-Wei Tseng, Qianchen Zhao, Yuxiao Zhou, Mark Gahagan, and Steven Swanson. 2016. Morpheus: creating application objects efficiently for heterogeneous computing. In Proceedings of the 43rd International Symposium on Computer Architecture (ISCA '16).

Morpheus


GPU
CPU
DRAM
YAML
TXT
CSV
FASTA
XML
{JSON}
SQL
LOG
GPX
SVG
HTML
POSTSCRIPT
SSD
Processor
Application objects
0xCAB23A
0x002FEA
0x1AA360
0x8CA520
0xBC614E
0x7BDE07
SSD

What's the benefit?

- Enabling P2P communication
- Reducing the data volume
- Lowering the energy consumption
- Bypassing host system overhead

The "Winning Formula" of Near-Data Processing

- The NDP computation can help offload computation
- The NDP computation can help reduce data volume
- The NDP computation can facilitate data preprocessing
- The NDP computation itself cannot be too slow

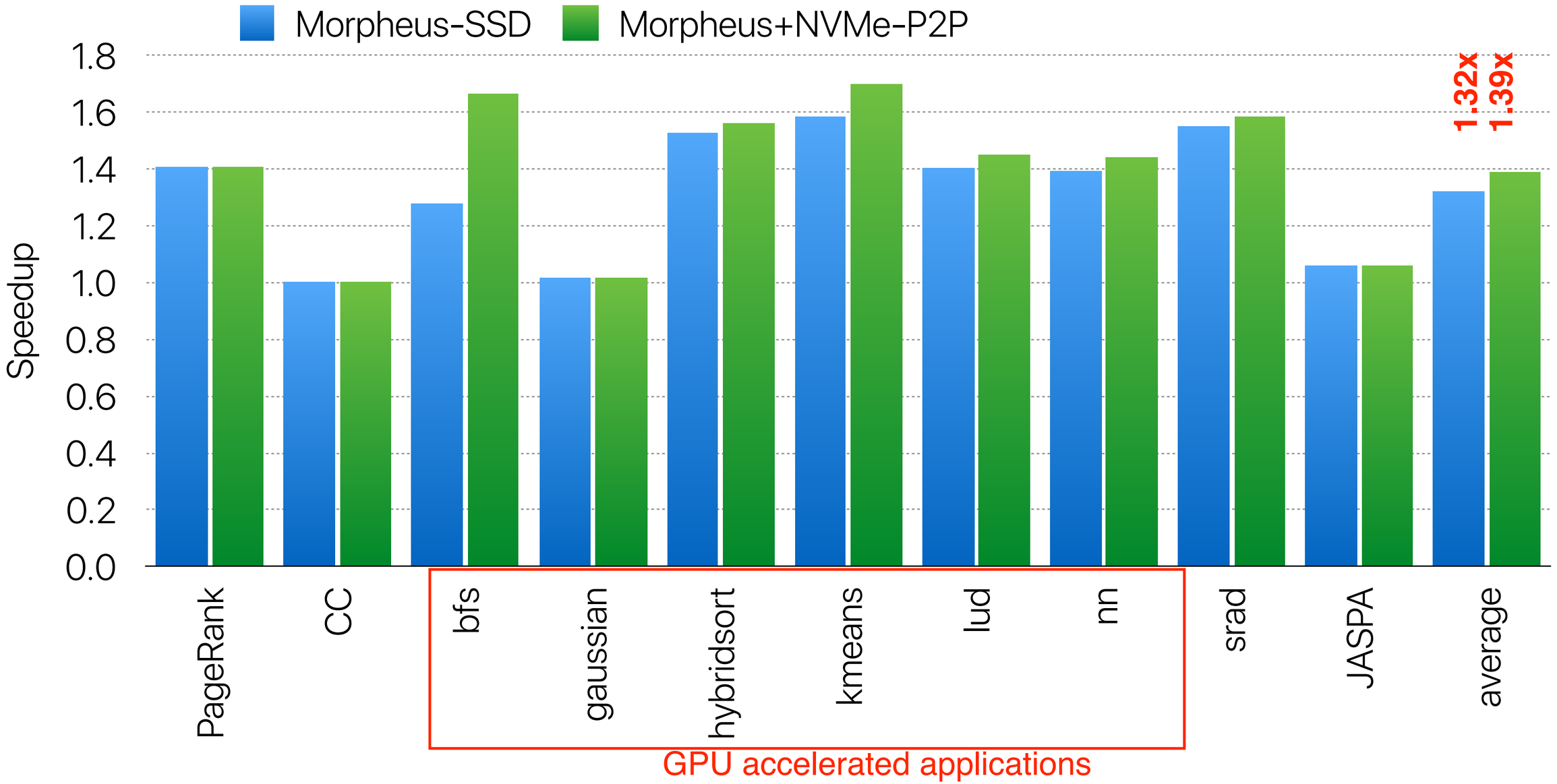
$$\frac{8GB}{8GBps} + \frac{8GB \times 2}{2GOPS} + \frac{2GB}{2.5GBps} + \frac{2GB \times 1}{1000GOPS} = 2.4$$

$$\frac{DataVolume_{raw}}{Bandwidth_{device}} + \frac{DataVolume_{raw} \times ComputationIntensity_{NDP}}{ComputationThroughput_{device}} + \frac{DataVolume_{afterNDP}}{Bandwidth_{device2host}} + \frac{DataVolume_{afterNDP} \times ComputationIntensity_{afterNDP}}{ComputationThroughput}$$

$$<= \frac{DataVolume_{raw}}{Bandwidth_{device2host}} + \frac{DataVolume_{raw} \times ComputationIntensity}{ComputationThroughput}$$

$$\frac{8GB}{2.5GBps} + \frac{8GB \times 2}{9GOPS} + \frac{8GB \times 1}{1000GOPS} = 3.2$$

Performance



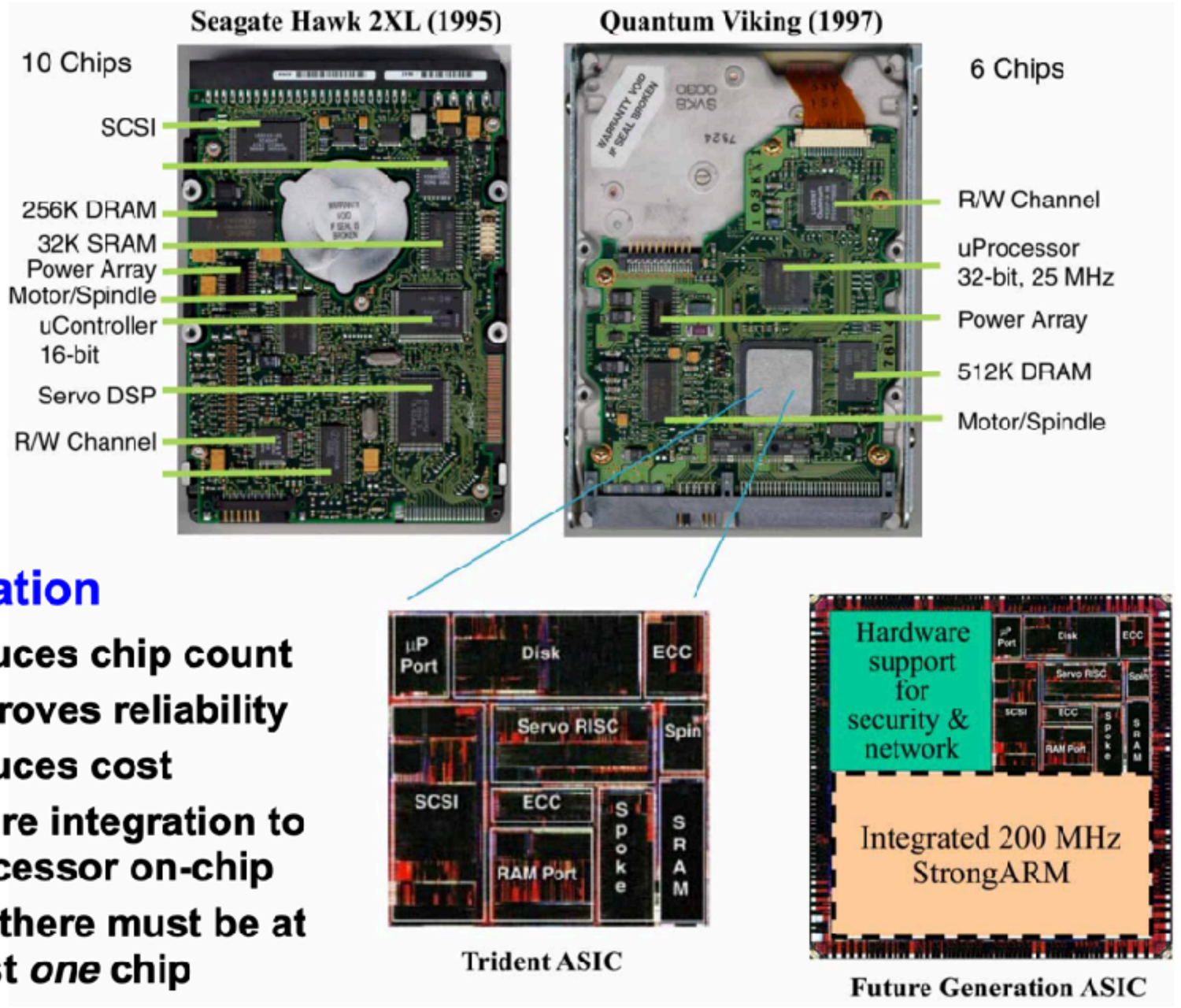
In-storage processing isn’t a new idea…

Active disks: programming model, algorithms and evaluation

Anurag Acharya, Mustafa Uysal, and Joel Saltz.
In Proceedings of the eighth international conference on Architectural support for programming languages and operating systems (ASPLOS VIII), 1998

http://www.cs.cmu.edu/~riedel/defense/talk99_ActiveDisksForDatabases.pdf

Evolution of Disk Drive Electronics

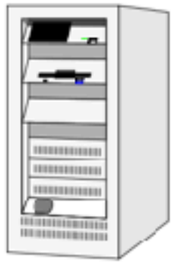


http://www.cs.cmu.edu/~riedel/defense/talk99_ActiveDisksForDatabases.pdf

Opportunity

TPC-D 300 GB Benchmark, Decision Support System

Database Server

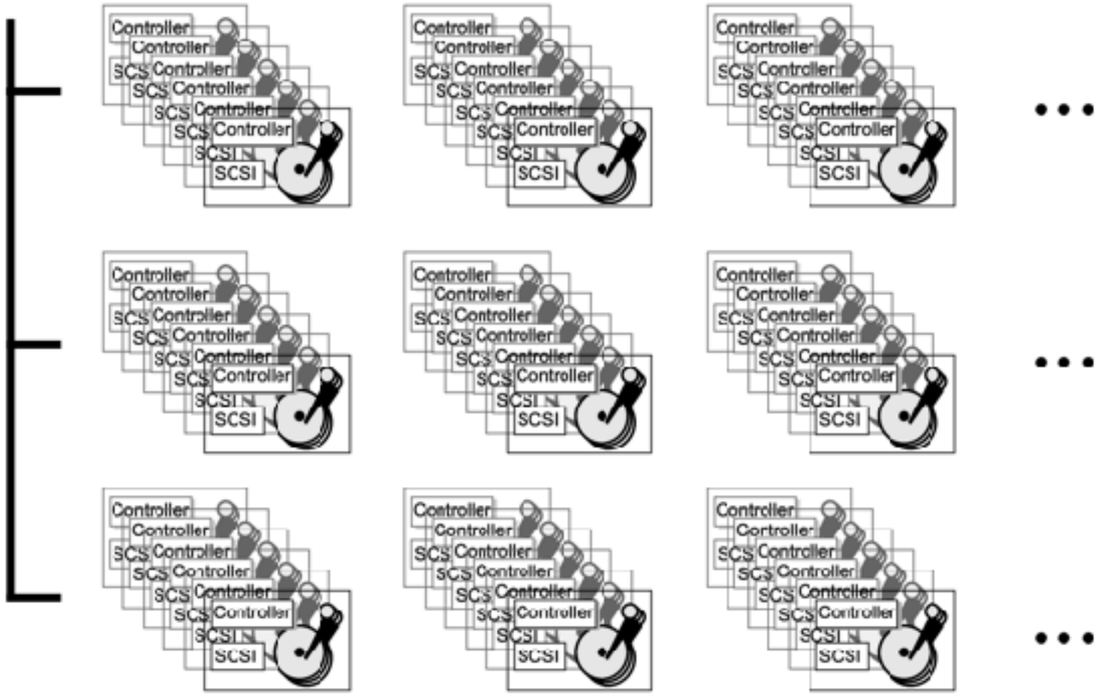


Digital AlphaServer 8400 = 7,344 total MHz
- 12 x 612 MHz 21164
- 8 GB memory
- 3 64-bit PCI busses — *3 x 266 = 798 MB/s*
- 29 FWD SCSI controllers — *29 x 40 = 1,160 MB/s*

Storage
- 520 rz29 disks
- 4.3 GB each
- 2.2 TB total

= 104,000 total MHz
(with 200 MHz drive chips)

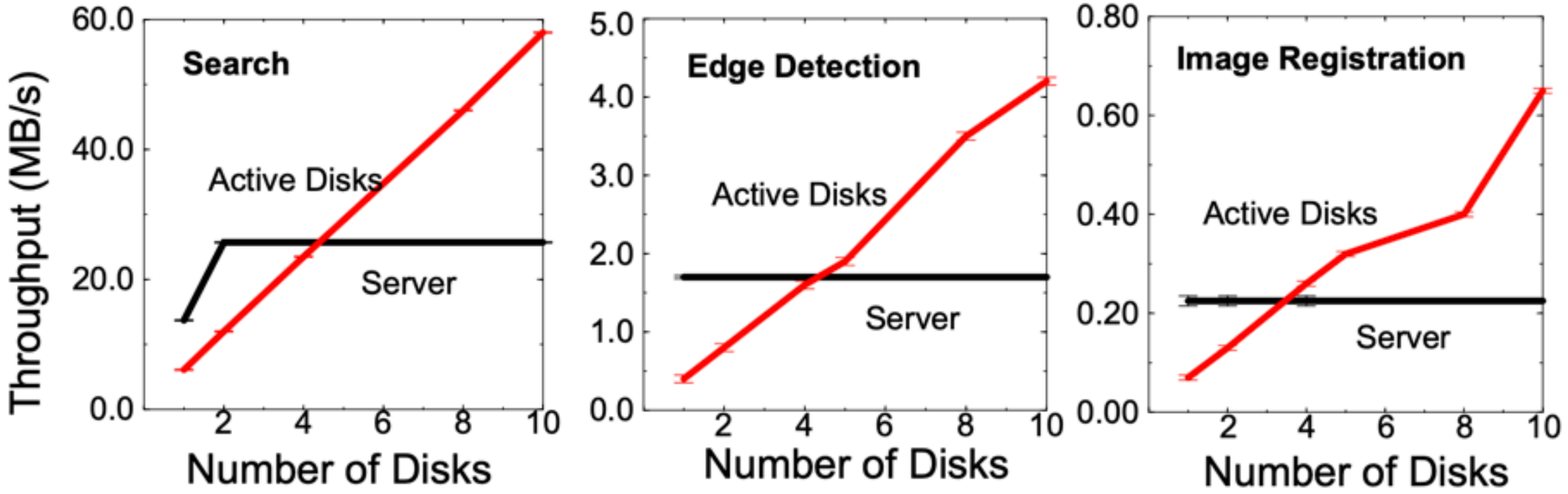
= 5,200 total MB/s
(at 10 MB/s per disk)



http://www.cs.cmu.edu/~riedel/defense/talk99_ActiveDisksForDatabases.pdf

Performance with Active Disks

application	input	computation (inst/byte)	throughput (MB/s)	memory (KB)	selectivity (factor)	bandwidth (KB/s)
Search	k=10	7	28.6	72	80,500	0.4
Frequent Sets	s=0.25%	16	12.5	620	15,000	0.8
Edge Detection	t=75	303	0.67	1776	110	6.1
Image Registration	-	4740	0.04	672	180	0.2



3:00

Why “active disks” did not work out in real practice?

Why isn’t ActiveDisk working?

Recap: The "Winning Formula" of In-Storage Processing

Conventional Model

ISP Model

$$\frac{DataVolume_{raw}}{Bandwidth_{device}} + \frac{DataVolume_{raw} \times ComputationIntensity_{NDP}}{ComputationThroughput_{device}} + \frac{DataVolume_{afterNDP}}{Bandwidth_{device2host}} + \frac{DataVolume_{afterNDP} \times ComputationIntensity_{afterNDP}}{ComputationThroughput}$$

$$<= \frac{DataVolume_{raw}}{Bandwidth_{device2host}} + \frac{DataVolume_{raw} \times ComputationIntensity}{ComputationThroughput}$$

The ISP processor must have efficient internal access to the device

The ISP computation cannot take too long

The ISP should reduce data volume

The ISP should offload the processor

Why Active Disks did not work out?

- Computation still dominates at that era
- Magnetic disks do not really have too much to do with “internal bandwidth”
- **The internal bandwidth is smaller than external bandwidth (need to show data)**
- Independent advances like distributed storage is similar to active disk
- New features at added cost may not be used by many and convincing system vendors to implement support is hard
- Business model — hard disk drives focus on low cost per unit of data

3:00

Can we summarize when will ISP/NDP work? Do you think there is a future of ISP/NDP?

Is ISP/NDP always a good idea?

When can ISP fail?

Conventional Model

ISP Model

- The ISP computation does not help offload computation
- The ISP computation does not help reduce data volume
- The ISP computation does not facilitate data preprocessing
- The ISP computation is compute intensive

Challenges of ISP

- Programming is still an issue
  - Do you want to program at the level of FPGAs?
- Applications
  - Low compute intensity
  - Volume reduction after pre-processing
- Hardware capability and cost
  - What kind of processors can we have in the storage device?
- Security
  - What if data is encrypted?

What if we just store them right at the beginning?

3:00

If you could define the data storage format, what’re you going to design for storing big data?

Ideas for big data storage?

Ideas for big data storage?

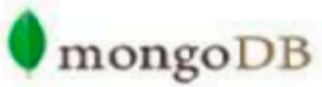
- Low overhead
  - Binary encoded
  - No compression
- Make no assumption to applications/algorithms
- Or at least fit most applications
  - Column-store? Row-store? Or?
  - Sparse? Dense?
- Easy for architectural optimizations (e.g., cache) to work
  - Properly chunked to fit page/cache block size

Modern Data Processing Stack

Relational Engines

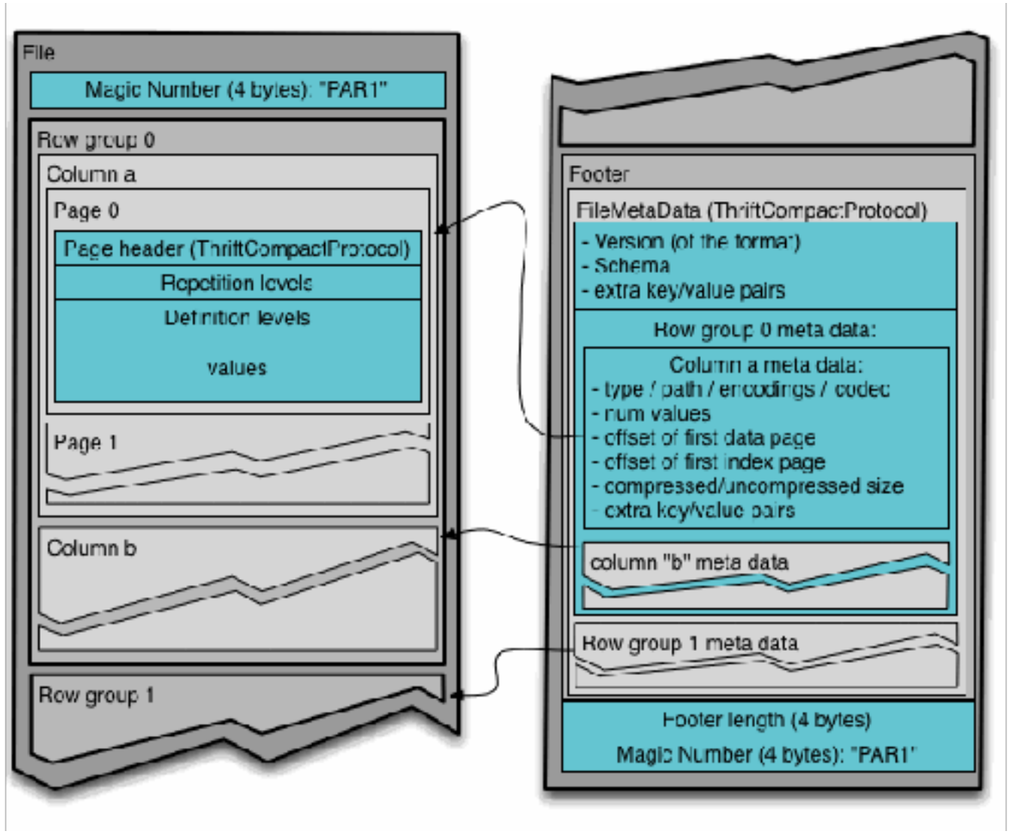
File Formats

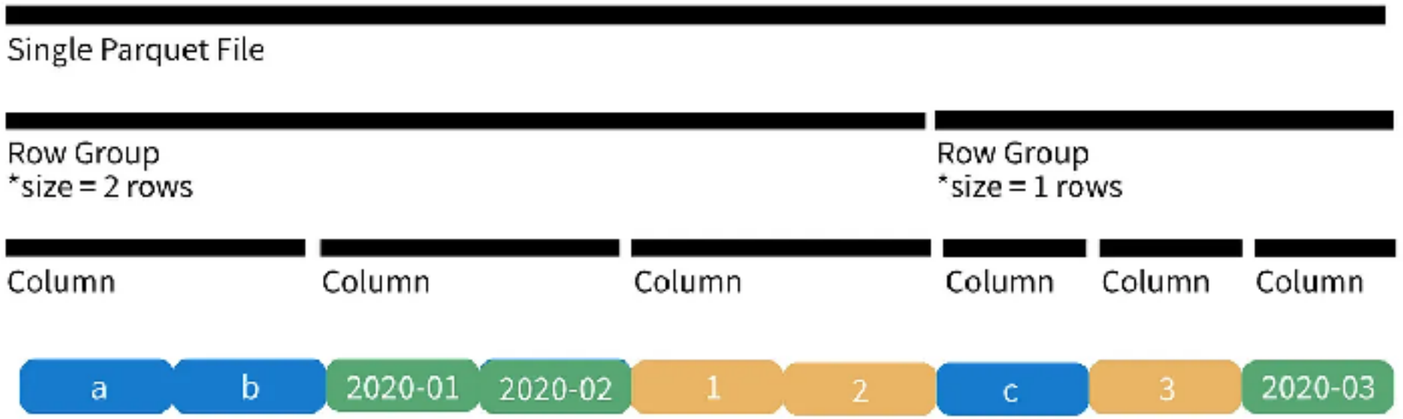
Distributed Storage



https://www.usenix.org/sites/default/files/conference/protected-files/atc18_slides_trivedi.pdf

Apache Parquet





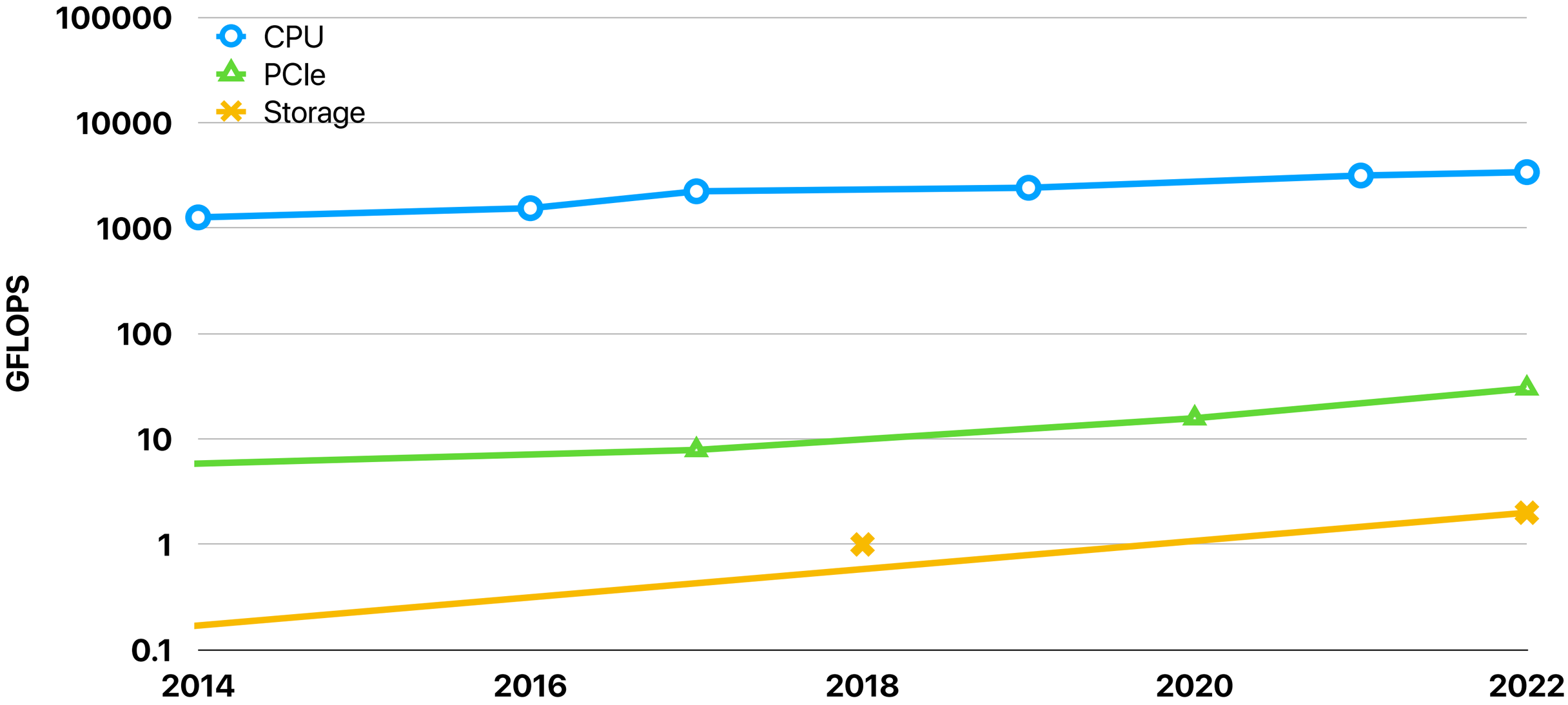
https://github.com/apache/parquet-format

3:00

Parquet is designed in 2013 — can you compare the computer systems at that time v.s. now?

Changes in the last decade?

CPU performance staggered



Datacenters prefer large chunks

The Google File System

Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung
Google*

2.5 Chunk Size

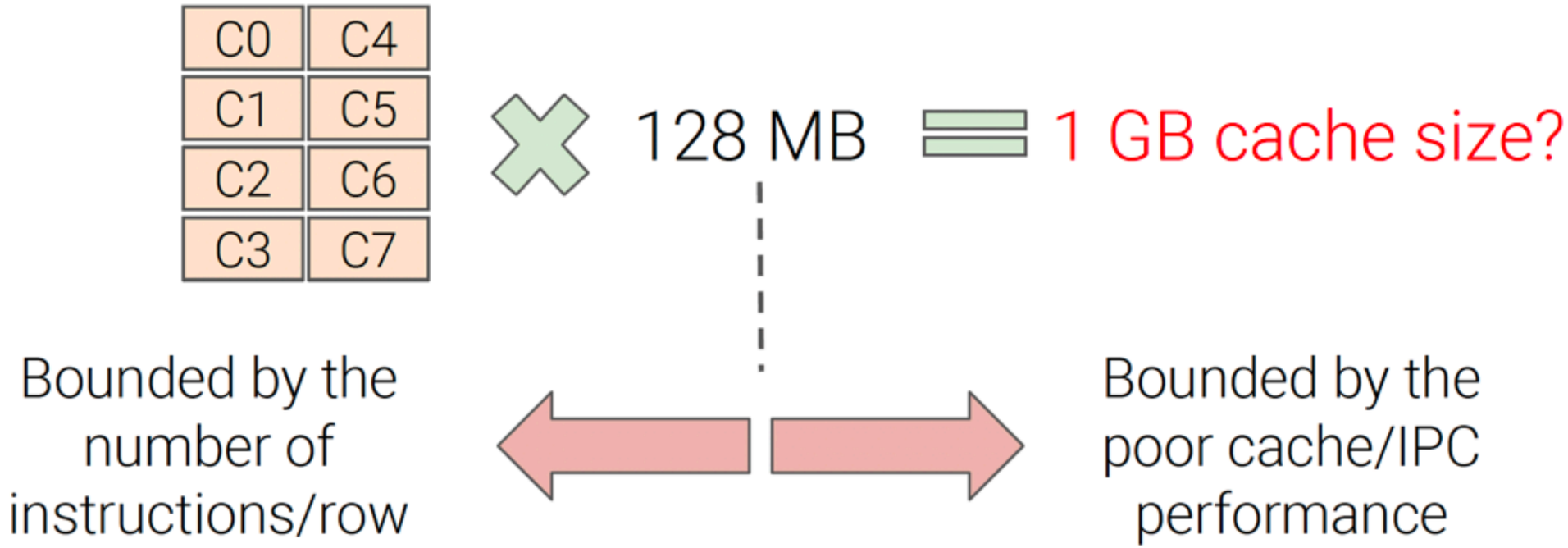
Chunk size is one of the key design parameters. We have chosen 64 MB, which is much larger than typical file system block sizes. Each chunk replica is stored as a plain Linux file on a chunkserver and is extended only as needed. Lazy space allocation avoids wasting space due to internal fragmentation, perhaps the greatest objection against such a large chunk size.

Finding a needle in Haystack: Facebook's photo storage

Doug Beaver, Sanjeev Kumar, Harry C. Li, Jason Sobel, Peter Vajgel,
Facebook Inc.
{doug, skumar, hcli, jsobel, pv}@facebook.com

Each Store machine manages multiple physical volumes. Each volume holds millions of photos. For concreteness, the reader can think of a physical volume as simply a very large file (100 GB) saved as '/hay/haystack_<logical volume id>'. A Store machine can access a photo quickly using only the id of the corresponding logical volume and the file offset at which the photo resides. This knowledge is the keystone of

But leads to bad CPU cache performance



https://www.usenix.org/sites/default/files/conference/protected-files/atc18_slides_trivedi.pdf

Multi-channel SSD — small random I/O ain't slow anymore

Flashtec™ NVMe2032 and NVMe2016 Controllers

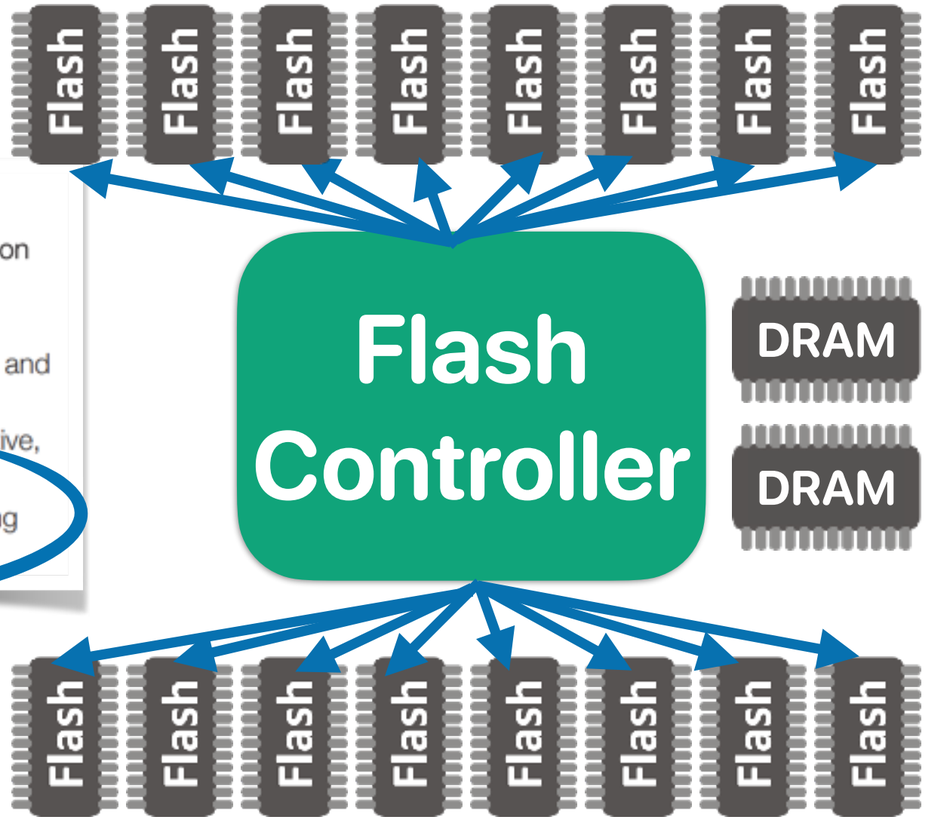
32- and 16-Channel PCIe Flash Controller Pr

Summary

The Flashtec™ 2nd generation NVMe Controller Family enables the
and data centers to realize the highest performance SSDs utilizing n
technologies. Combining world-class capacity and flexibility, the Flas
reliable choice. The Flashtec NVMe2032 and NVMe2016 controllers
Express (NVMe) host interface and are optimized for high-performan
operations, performing all flash management operations on-chip an
processing and memory resources.

Features

- Flashtec NVMe2032 controller can achieve up to 1 million random read IOPS on 4 KB operations
- Up to 20 TB Flash capacity using 256 GB Flash
- SLC, MLC, Enterprise MLC, and TLC Flash with toggle and ONFI interface
- PCIe Gen 3 x8 or dual independent PCIe Gen 3 x4 (active, active/standby) host interface
- 16 and 32 independent Flash channels, each supporting up to 8 CE



Each ~500MB/sec
8 channels == 4GB/sec
16 channels == 16 GB/sec

Micron

- Organization
  - Page size x8: 18,592 bytes (16,384 + 2208 bytes)
  - Block size: 2304 pages, (36,864K + 4968K bytes)
  - Plane size: 4 planes x 504 blocks
  - Device size: 512Gb: 2016 blocks; 1Tb: 4032 blocks; 2Tb: 8064 blocks; 4Tb: 16,128 blocks; 8Tb: 32,256 blocks

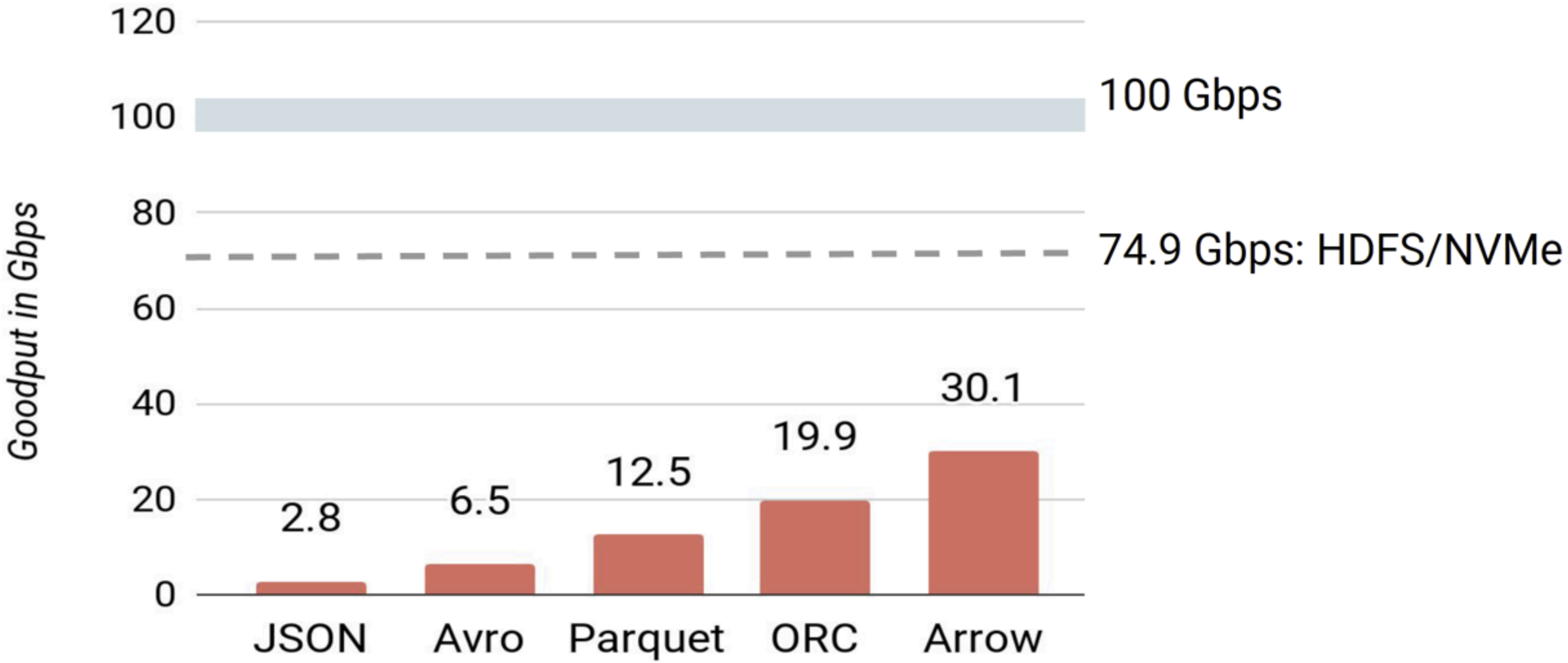
TLC 512Gb-8Tb NAND – Array Characteristics

is suspended or ongoing					
READ PAGE operation time without/with V_{PP}	^tR	57/41	60	µs	2, 12
SNAP READ operation time	^tRSNAP	27	37	µs	
Cache read busy time	^tRCBSY	11	60	µs	2, 8, 12

Changes in the last decade

- CPU speed v.s. I/O?
- Sequential I/O v.s. Random I/O?
- Remote I/O?
- Metadata lookup?
- Runtime systems

The deserialization performance on popular formats



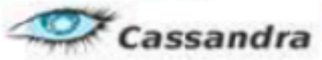
https://www.usenix.org/sites/default/files/conference/protected-files/atc18_slides_trivedi.pdf

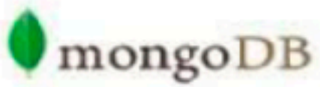
Modern Data Processing Stack

Relational Engines

File Formats

Distributed Storage



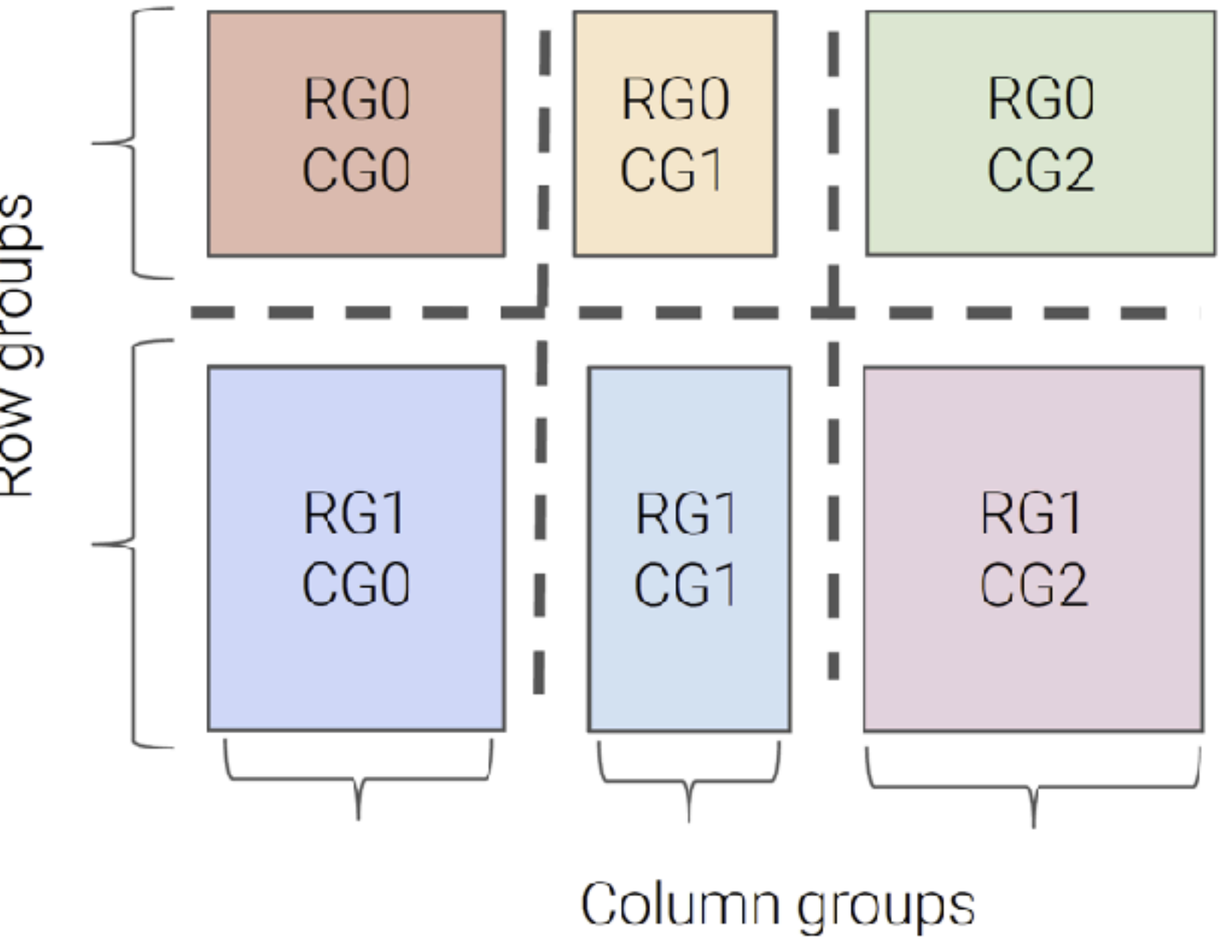


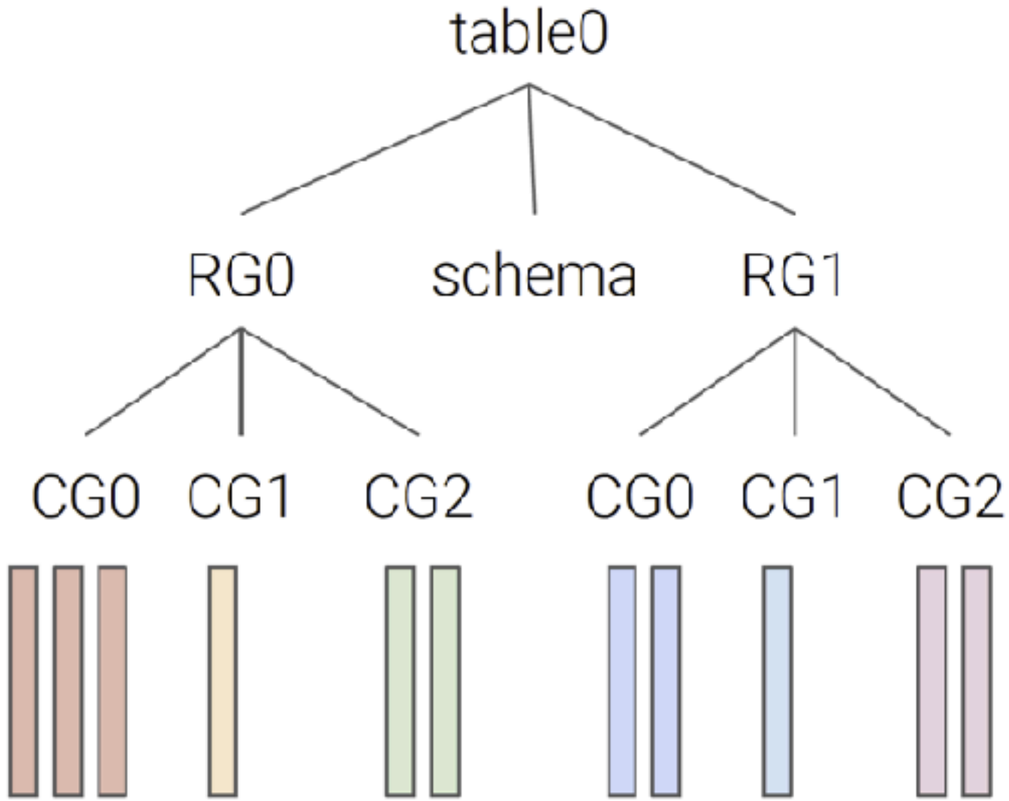
https://www.usenix.org/sites/default/files/conference/protected-files/atc18_slides_trivedi.pdf

Albis

- Albis - A file format to store relational tables for read-heavy analytics workloads
  - Supports all basic primitive types with data and schema
  - Nested schemas are flattened and data is stored in the leaves
- Three fundamental design decisions:
  - Avoid CPU pressure, i.e., no encoding, compression, etc.
  - Simple data/metadata management on the distributed storage
  - Carefully managed runtime - simple row/column storage with a binary API

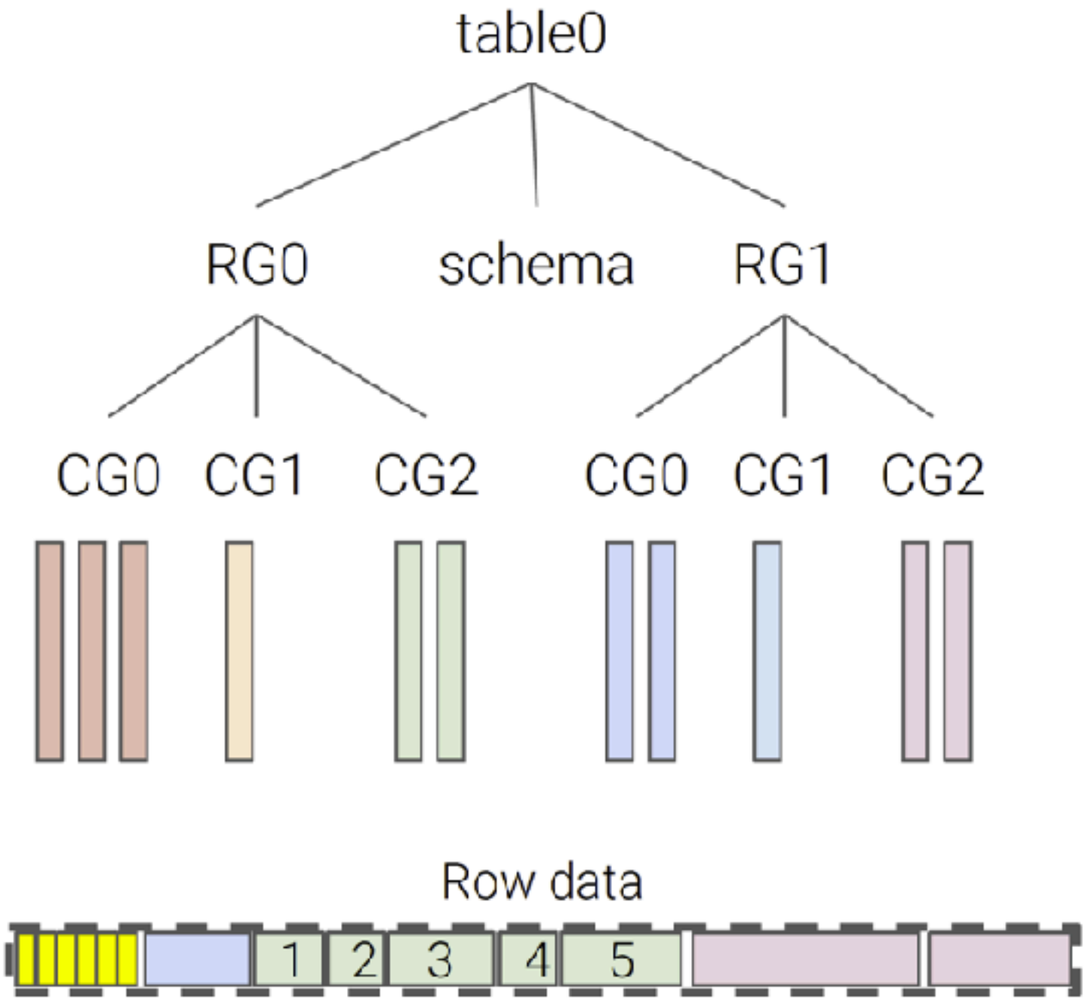
Not just row groups, but also column groups





https://www.usenix.org/sites/default/files/conference/protected-files/atc18_slides_trivedi.pdf

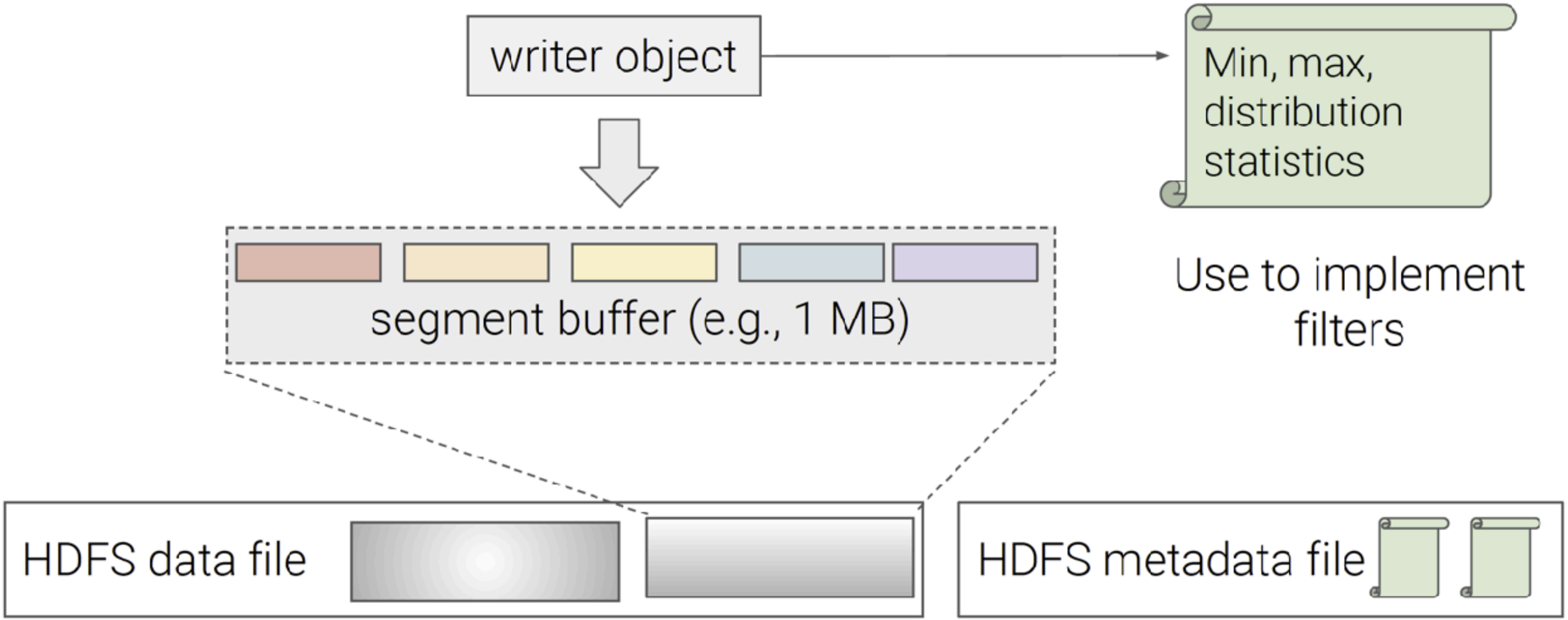
Reading a row



1. Read schema file
2. Check projection to figure out which files to read
    a. Complete CGs
    b. Partial CGs
3. Evaluate filters to skip segments
4. Materialize values
    a. Skip value materialization in partial CG reads

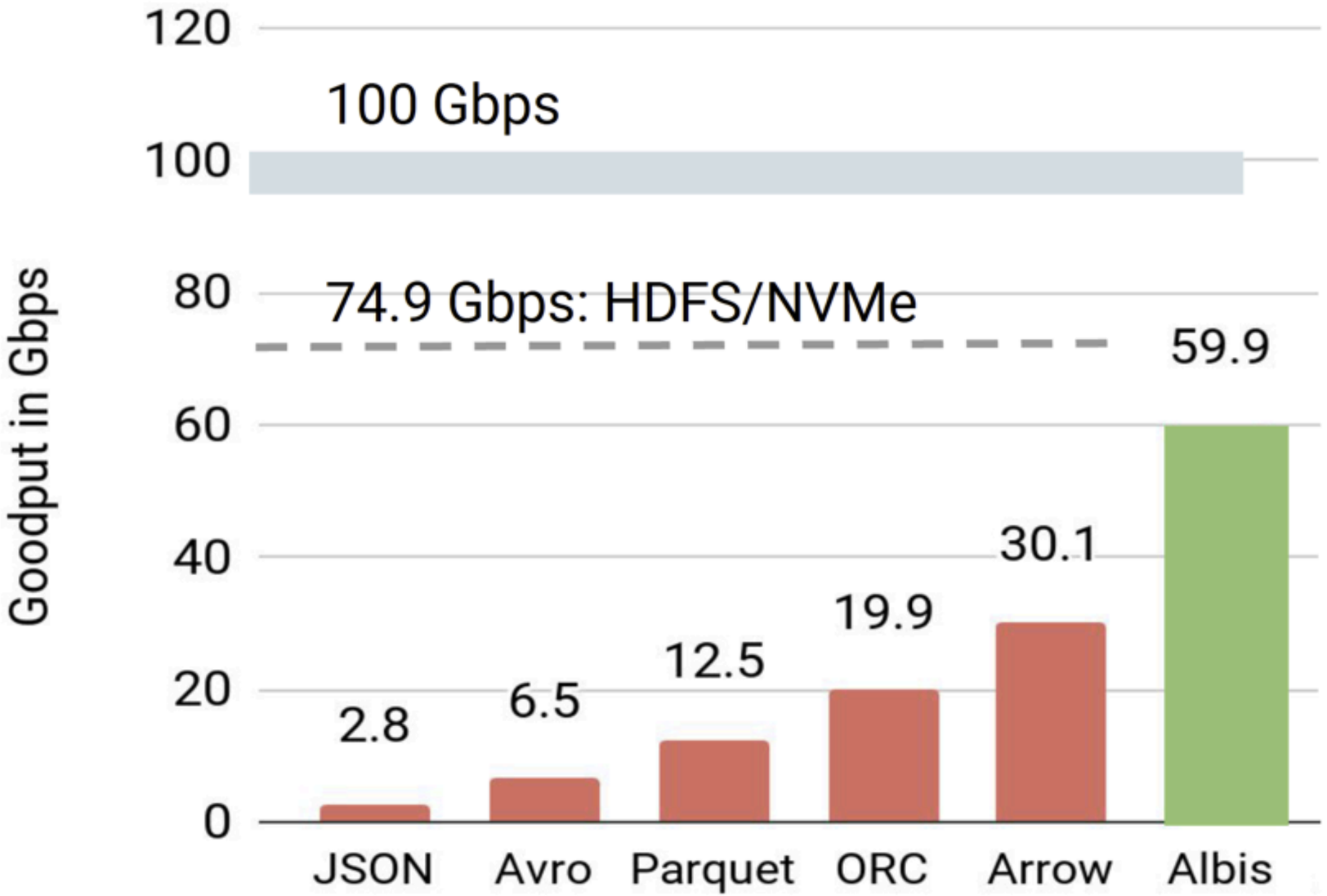
https://www.usenix.org/sites/default/files/conference/protected-files/atc18_slides_trivedi.pdf

Writing a row



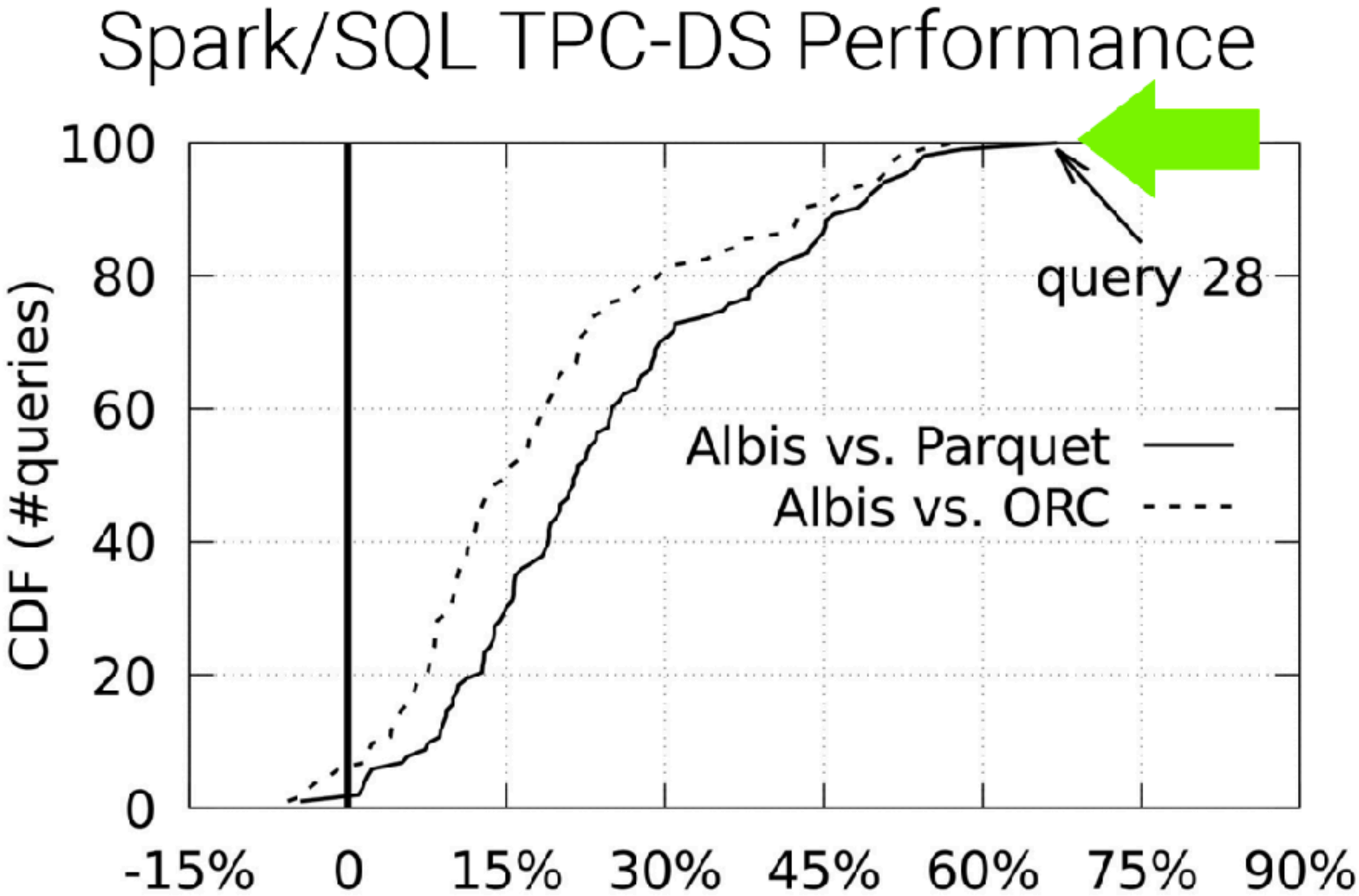
https://www.usenix.org/sites/default/files/conference/protected-files/atc18_slides_trivedi.pdf

Performance of Albis



https://www.usenix.org/sites/default/files/conference/protected-files/atc18_slides_trivedi.pdf

Albis v.s Parquet

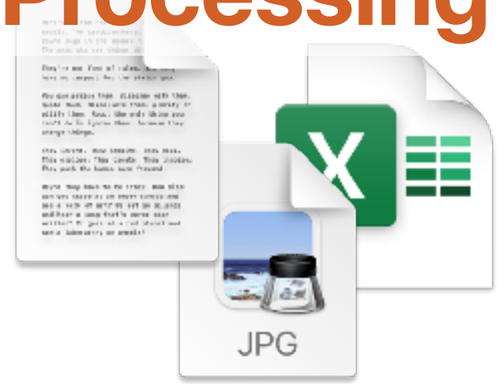


https://www.usenix.org/sites/default/files/conference/protected-files/atc18_slides_trivedi.pdf

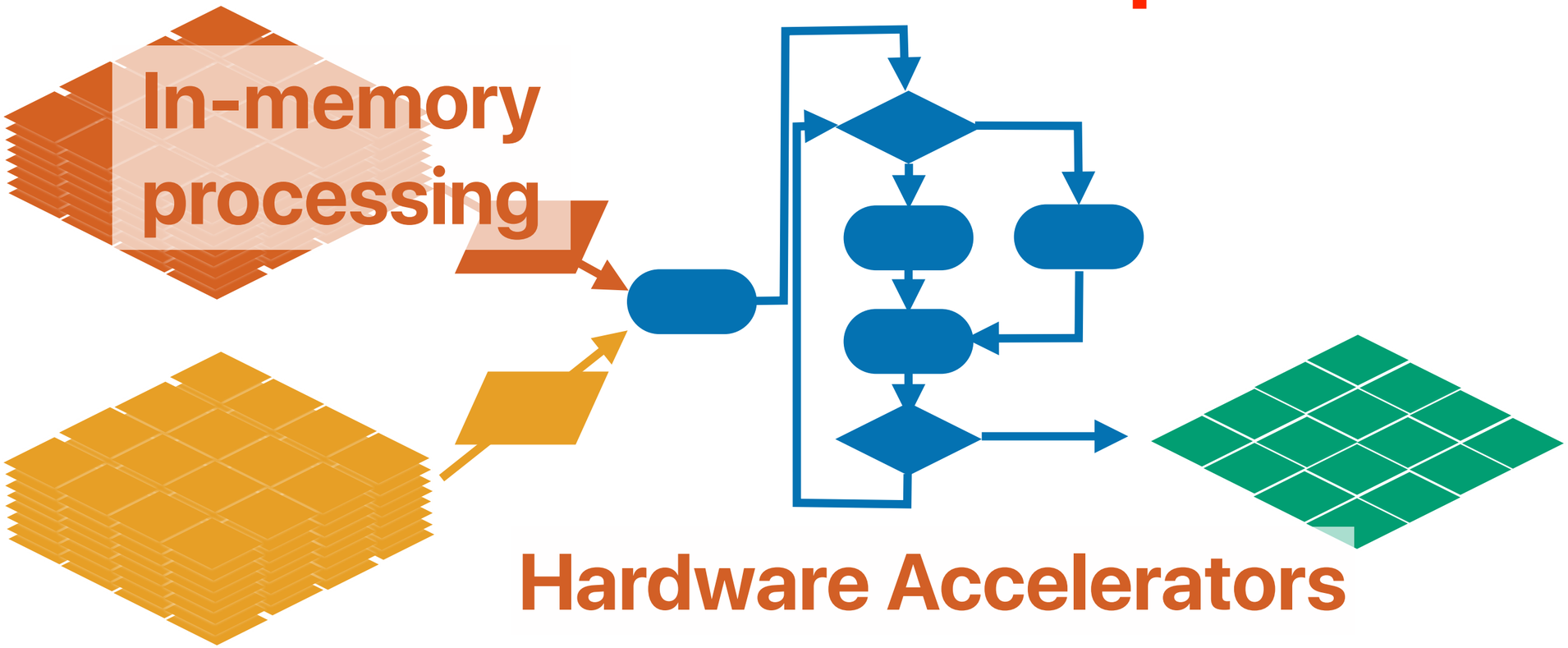
What have we learned?

Recap: what have we learned this quarter

In-Storage Processing



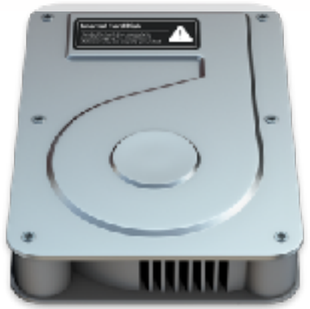
In-memory processing



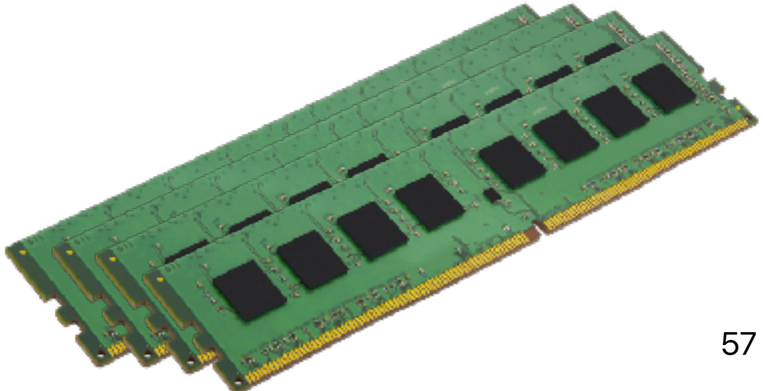
Hardware Accelerators

Right data formats

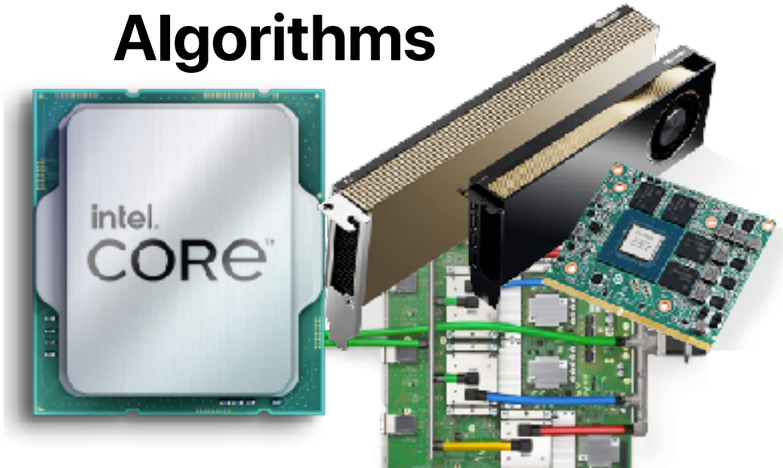
Source "data"



"Data" structures



Algorithms

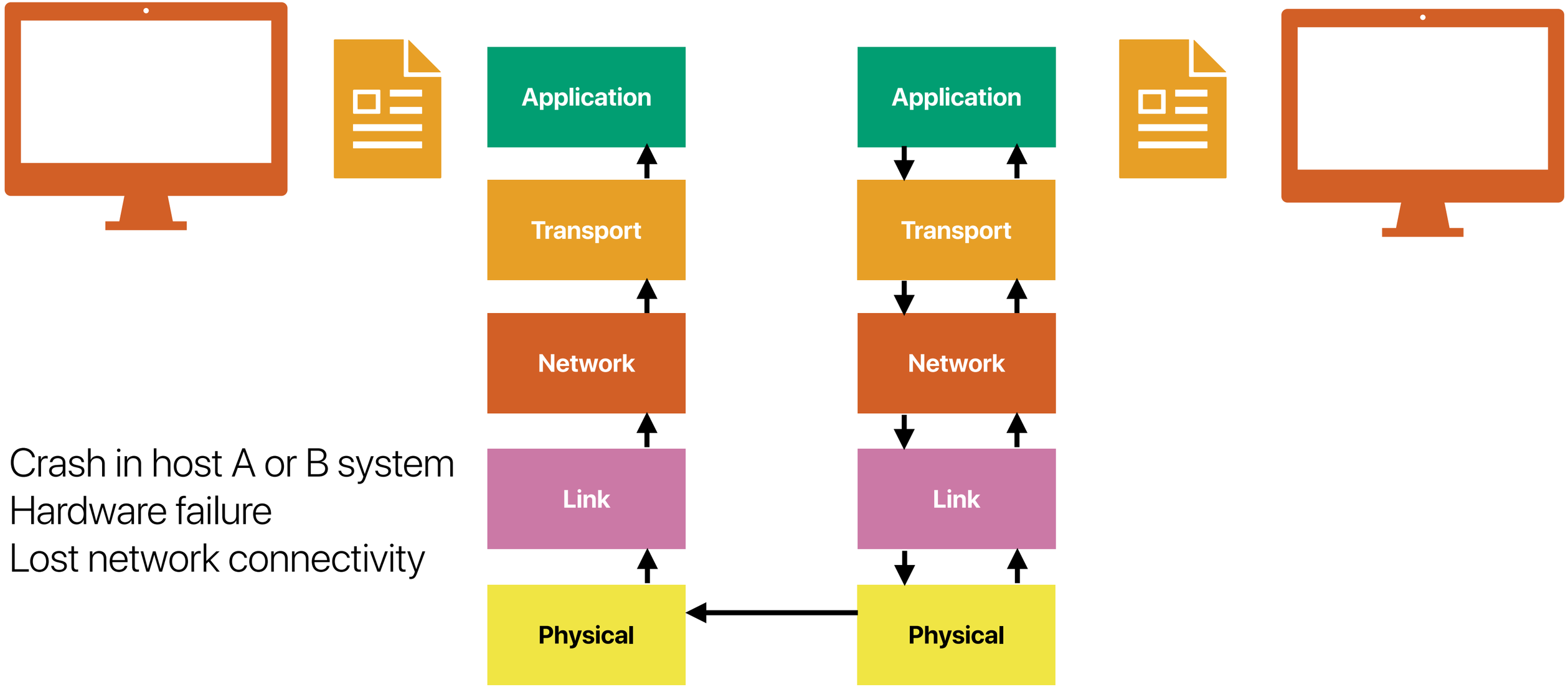


Result

e2e argument

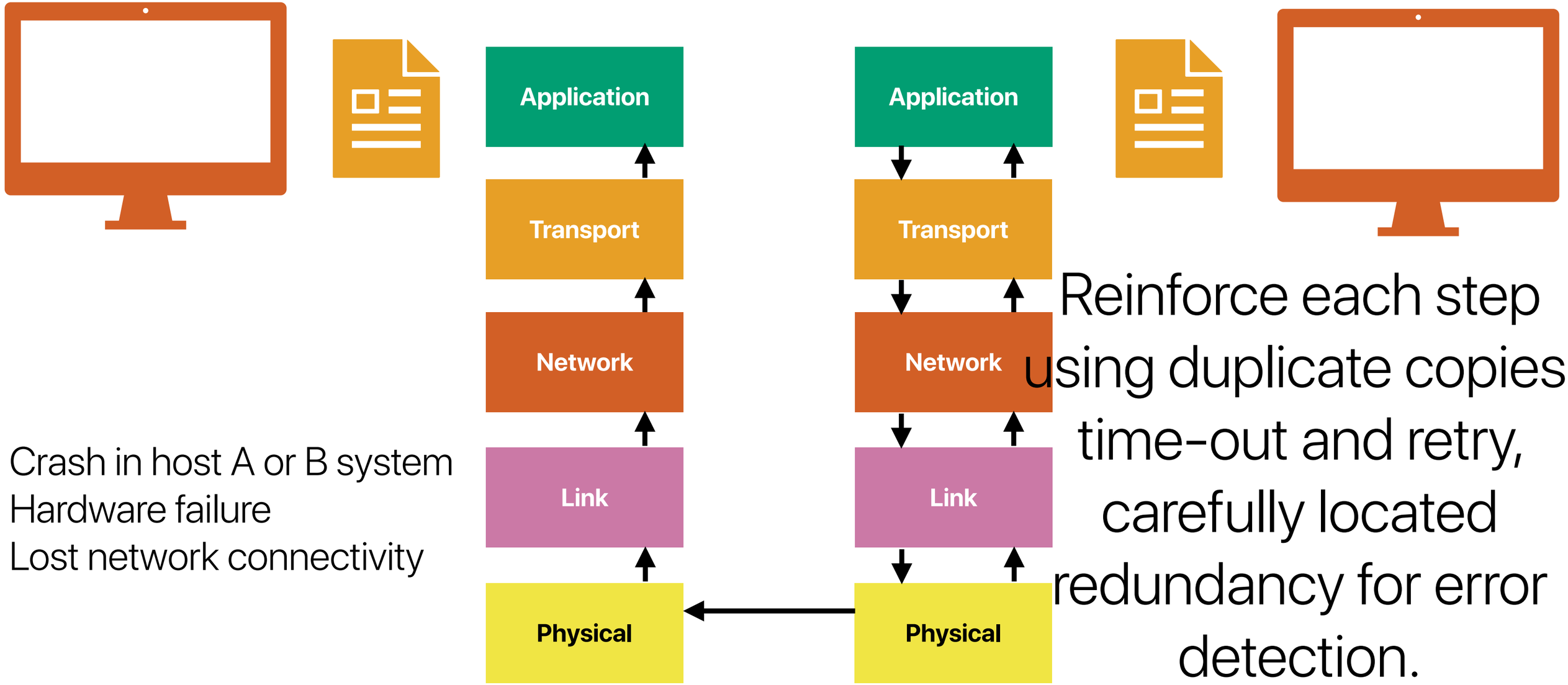
The function in question can completely and correctly be implemented only with the knowledge and help of the application standing at the endpoints of the communication system. Therefore, providing that questioned function as a feature of the communication system itself is not possible. (Sometimes an incomplete version of the function provided by the communication system may be useful as a performance enhancement.)

Example — file transfer



Crash in host A or B system
Hardware failure
Lost network connectivity

File transfer: current approach



Crash in host A or B system
Hardware failure
Lost network connectivity

Reinforce each step using duplicate copies, time-out and retry, carefully located redundancy for error detection.

e2e argument

- The application program follows the simple steps in transferring the file.
- As the final step, host B will recalculate the checksum for the transferred file.
- Then host B will send this value to be compared with the original value at host A.
- If this value doesn’t mach, the file will be resend again to host A.

Why the e2e argument?

- If we consider a network that is somewhat unreliable, dropping one packet of each hundred packet sent
- Using the above outline strategy the above network will do a bad job.
- Clearly , some effort at the lower levels to improve the network reliability will significantly improve the performance.
- The key idea here: The lower level need not to provide “perfect” reliability
- E2E check of the file transfer application must be implemented no matter how reliable the communication system become.
- The amount of effort to put into reliability within the data communication system is seen to be an engineering trade-off based on performance.

Why the e2e argument?

- Using performance to justify placing functions in low-level subsystem must be done carefully since performing the function in the low level may cost more because:
  - Some application which use the same low level subsystem and doesn’t need this function will pay for it anyway.
  - Low-level system may not have as much information as the higher level, so it cannot do the job efficiently.

Summary of e2e arguments

- All applications might not use the service
- Trade-off between performance and cost
- Combine network and application information to optimize performance

3:00

What idea(s) we've learned this quarter is/are against the e2e argument?
What idea(s) we've learned this quarter supporting the e2e argument?

Ideas against the e2e argument

Ideas against the e2e argument

- In-memory processing
  - What the e2e argument hits?
    - Programming efforts and inefficiency from the lack of high-level information
  - What the e2e argument gets wrong?
- In-storage processing
  - What the e2e argument hits?
    - Lack of knowledge in file systems and must consult host computers a lot
  - What the e2e argument gets wrong?

Hints for computer system design

Butler W. Lampson
Computer Science Laboratory Xerox Palo Alto Research Center

Hints for computer system design

Why?	*Functionality* Does it work?	*Speed* Is it fast enough?	*Fault-tolerance* Does it keep working?
Where?			
Completeness	Separate normal and worst case	Shed load End to end Safety first	End to end
Interface	Do one thing well: Don't generalize Get it right Don't hide power Use procedure arguments Leave it to the client Keep basic interfaces stable Keep a place to stand	Make it fast Split resources Static analysis Dynamic translation	End-to-end Log updates Make actions atomic
Implementation	Plan to throw one away Keep secrets Use a good idea again Divide and conquer	Cache answers Use hints Use brute force Compute in background Batch processing	Make actions atomic Use hints

Hints for computer system design

Why?	*Functionality* Does it work?	*Speed* Is it fast enough?	*Fault-tolerance* Does it keep working?
Where?			
Completeness	Separate normal and worst case	Shed load End to end	End to end
		Safety first	
Interface	Do one thing well: Don't generalize Get it right Don't hide power Use procedure arguments Leave it to the client Keep basic interfaces stable Keep a place to stand	Make it fast Split resources Static analysis Dynamic translation	End-to-end Log updates Make actions atomic
Implementation	Plan to throw one away Keep secrets Use a good idea again Divide and conquer	Cache answers Use hints Use brute force Compute in background Batch processing	Make actions atomic Use hints

Amdahl's Law

correctness matters

Hardware accelerators

FTL

Caching

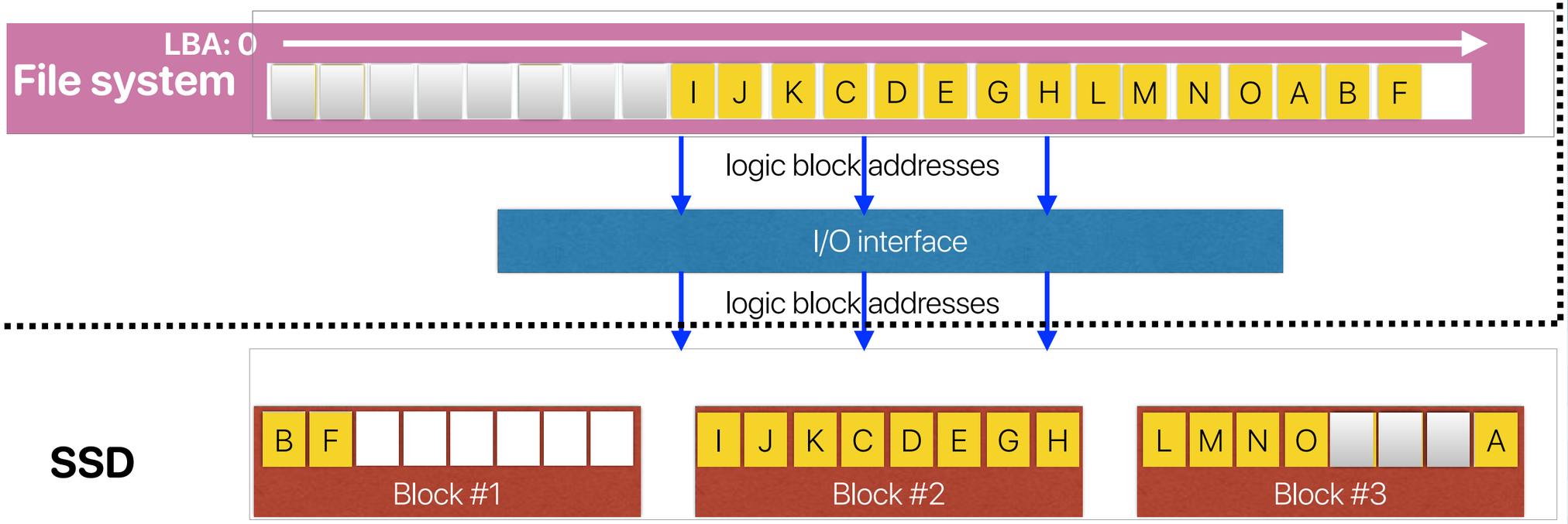
Pros/Cons of Layering

- Good
  - Ease of Debugging (T.H.E. by Dijkstra)
- Bad
  - Performance
  - Sub-optimal optimizations


R
Perl
Java
C
C++
python
PYTORCH
TensorFlow
cuDNN
NVIDIA
CUDA
Mac OS
Linux
Windows
ANDROID
? ? ? ?
GPU
Accelerators
FPGA
intel
Xeon
processor
CPU
SSD
NIC
DRAM

? ? ? ?

What will happen if the FS wants to perform GC?



We could have avoided writing the stale A, B, F if they are coordinated!

invalid

valid

free

FTL mapping table

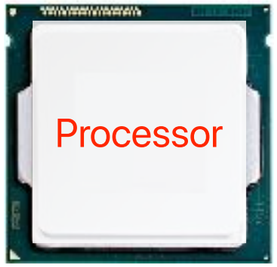
LBA	block #	page #
0	-	-
1	-	-
2	-	-
3	-	-
4	-	-
5	-	-
6	-	-
7	-	-
8	2	0
9	2	1
10	2	2
11	2	3
12	2	4
13	2	5
14	2	6
15	2	7
16	3	0
17	3	1
18	3	2
19	3	3
20	3	7
21	1	0
22	1	1
23	-	-

All problems in computer science can be solved by another level of indirection

–David Wheeler

...except for the problem of too many layers of indirection.

When the conventional interface meets modern applications



ORACLE NOSQL DATABASE

APACHE HBASE

Google Cloud BigTable

keys

Mapping Tables

Transaction support

Caches

Logs

files, offsets

File systems

logic block addresses

Read/Write

logic block addresses

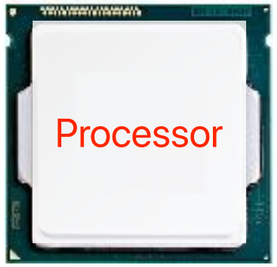
Flash translation layer

physical addresses

Address mapping

Log

When the conventional interface meets modern applications



ORACLE NOSQL DATABASE

APACHE HBASE

Google Cloud BigTable

keys

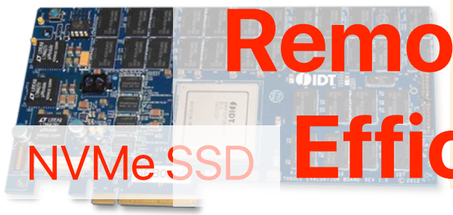
Mapping Tables | Transaction support | Caches | Logs

files, offsets

File systems

logic block addresses

Key-value Store

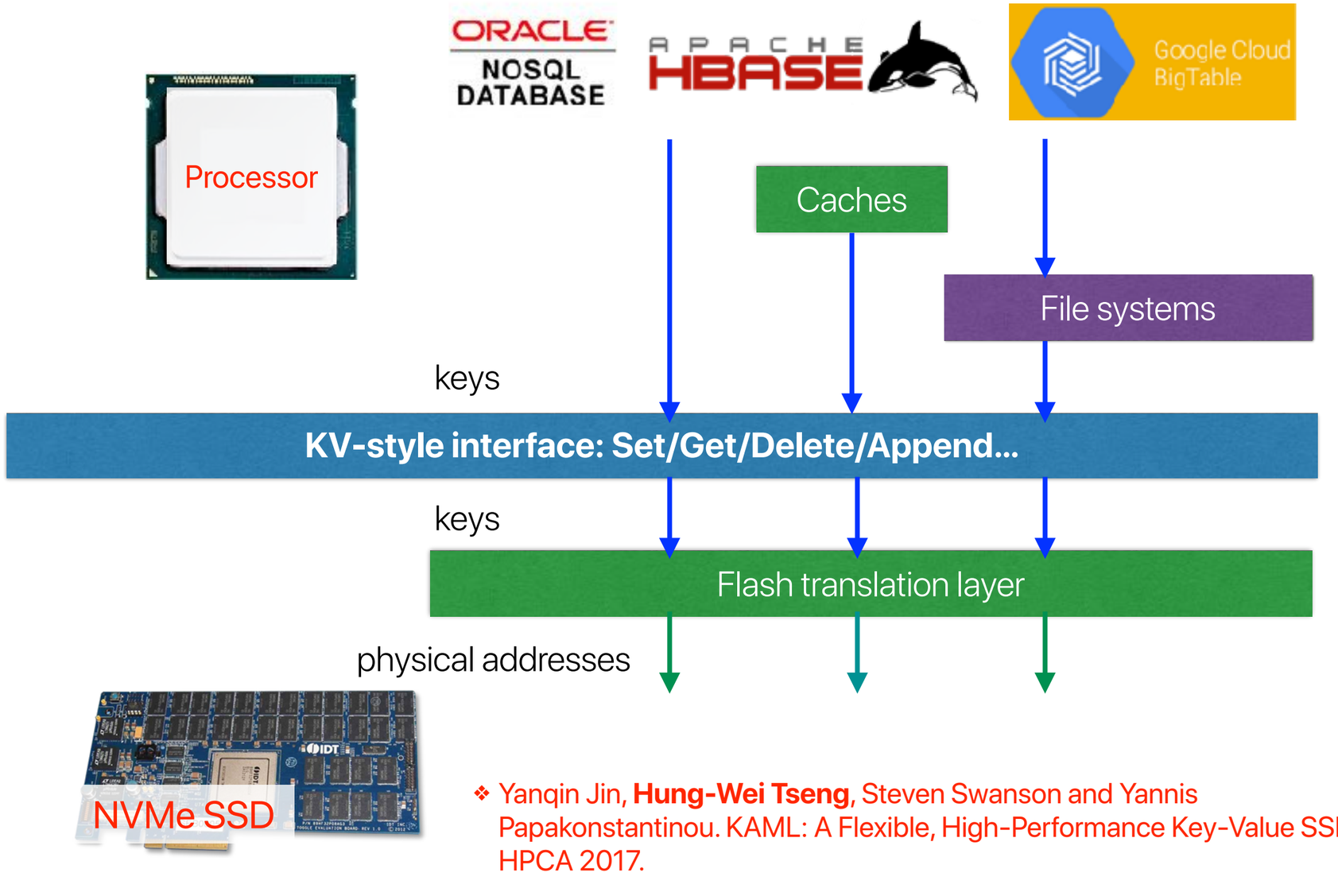


Avoid multiple indirections

Remove duplicated optimizations

Efficient data synchronization

KAML: A Flexible, High-Performance Key-Value SSD



- Yanqin Jin, **Hung-Wei Tseng**, Steven Swanson and Yannis Papakonstantinou. KAML: A Flexible, High-Performance Key-Value SSD. In HPCA 2017.

Announcement

- Course evaluations — help us keeping the class great!
- What to expect in the final presentation
  - Brief recap of the why and what — 5 minutes
  - What concrete experimental results have been measured — 7 minutes
    - You need to have convincing experimental setup
    - You need to have several result figures (i.e., bar charts) or analysis on why your ideas do not work
    - What insights have you learned that would guide future research

Electrical
Computer Science
Engineering

277

ありがとう!